ADVANCED DATABASE SYSTEMS

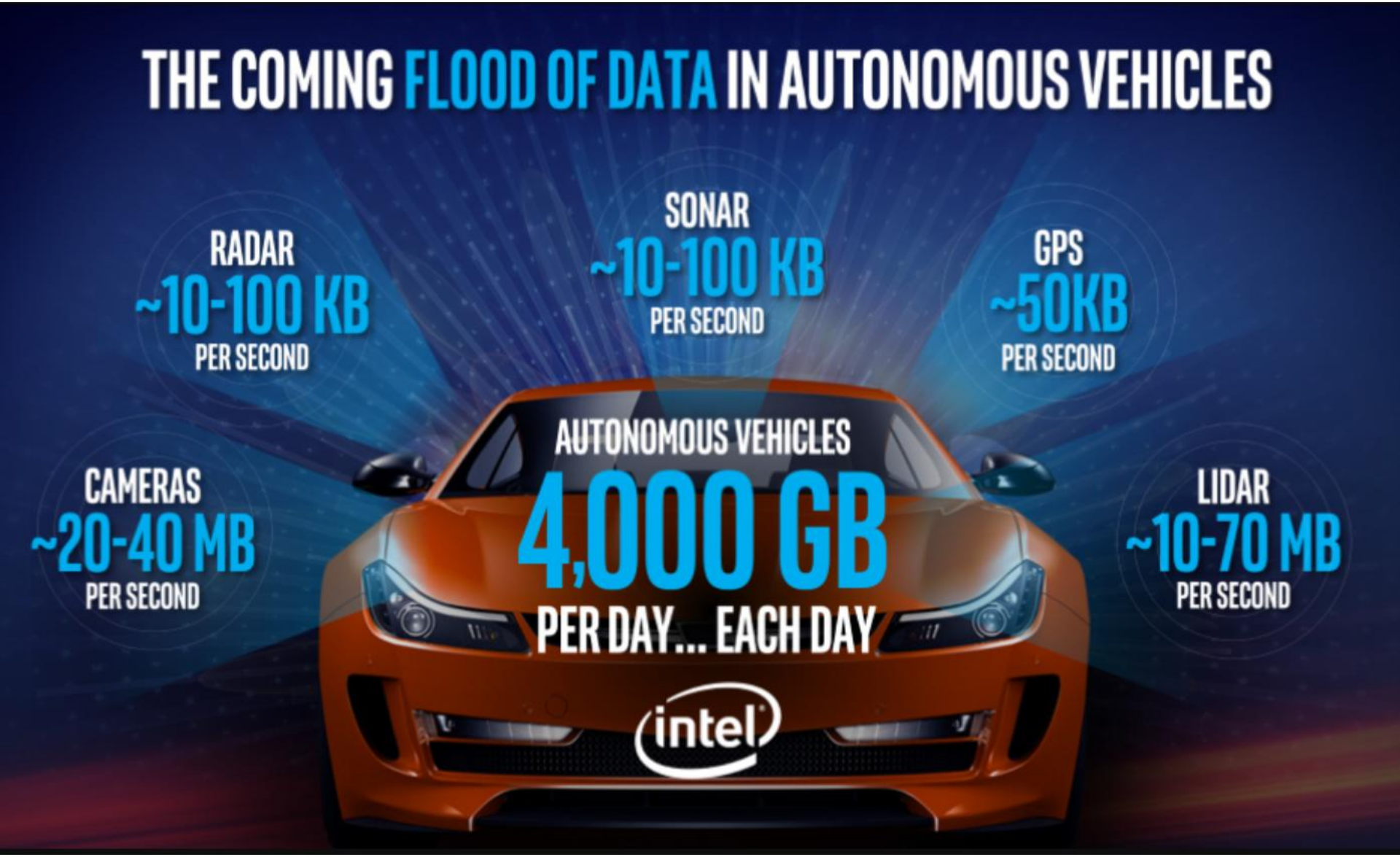
WEEK 1

Slides credit: S. Shivakumar, Silberschatz, Korth, Sudarshan

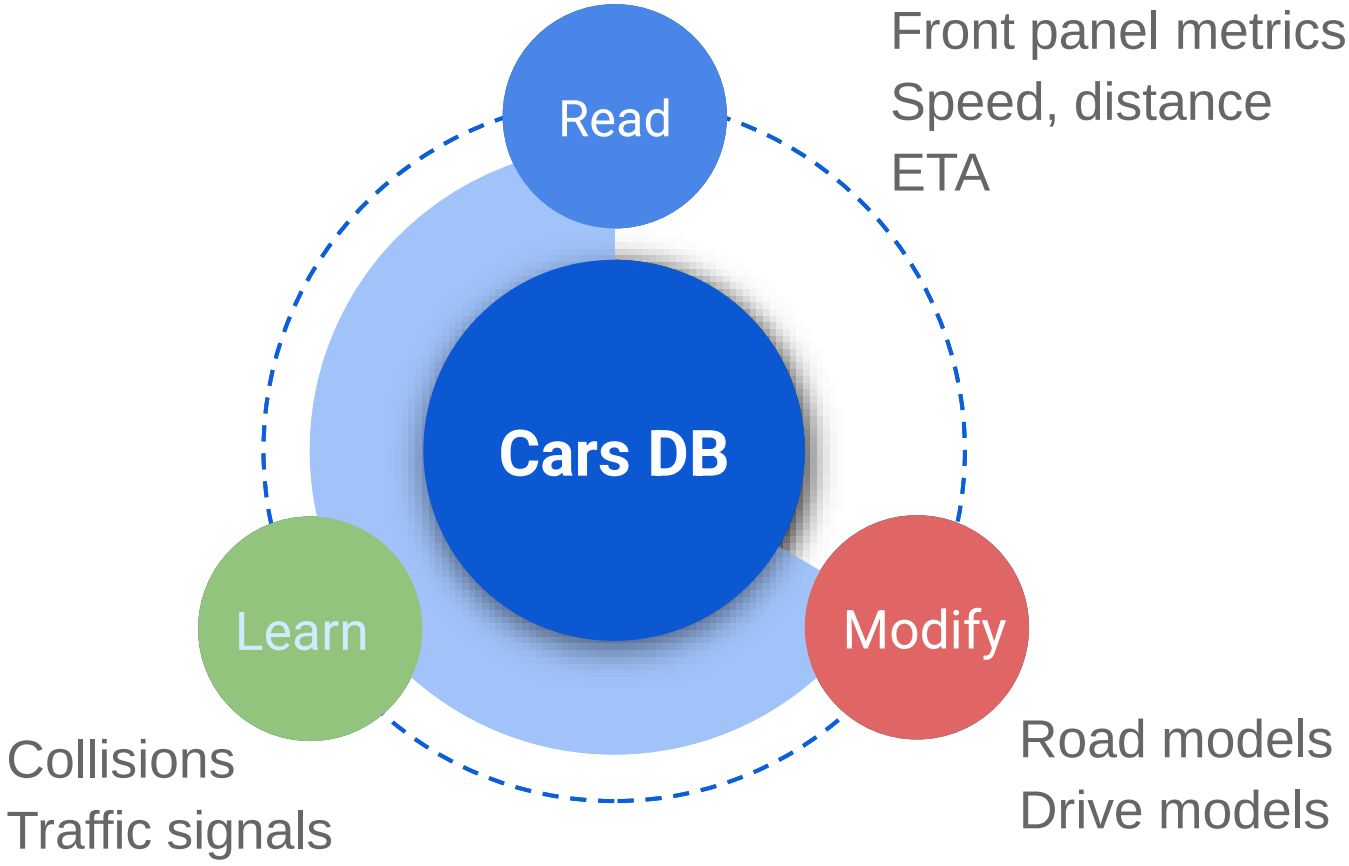
Goals of a DBMS

- Data storage & access
  - Efficient
  - Convenient
  - Safe
  - Multi-user
  - Scalable
  - Persistent

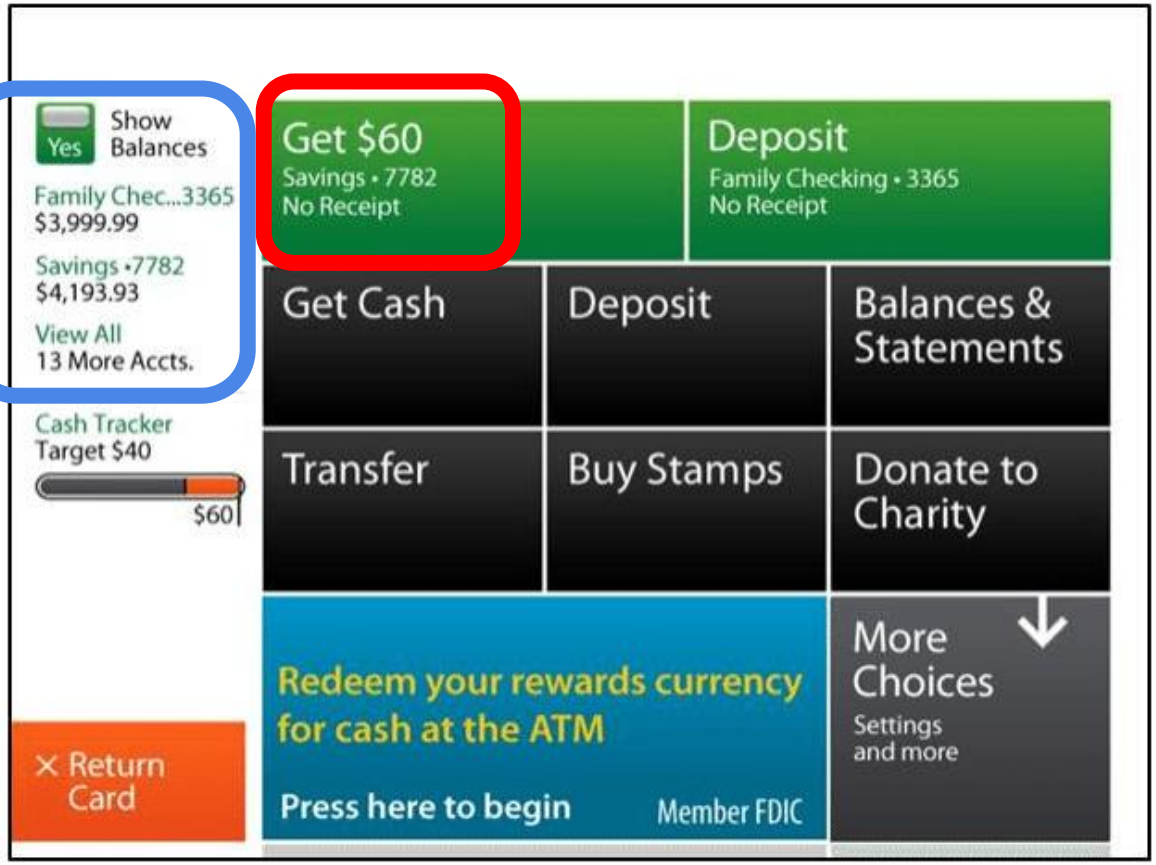
Example: Self Driving Cars


THE COMING FLOOD OF DATA IN AUTONOMOUS VEHICLES
RADAR
~10-100 KB
PER SECOND
SONAR
~10-100 KB
PER SECOND
GPS
~50KB
PER SECOND
CAMERAS
~20-40 MB
PER SECOND
AUTONOMOUS VEHICLES
4,000 GB
PER DAY... EACH DAY
LIDAR
~10-70 MB
PER SECOND
intel

Unpack Cars DB



Example
Unpack ATM DB: Transaction



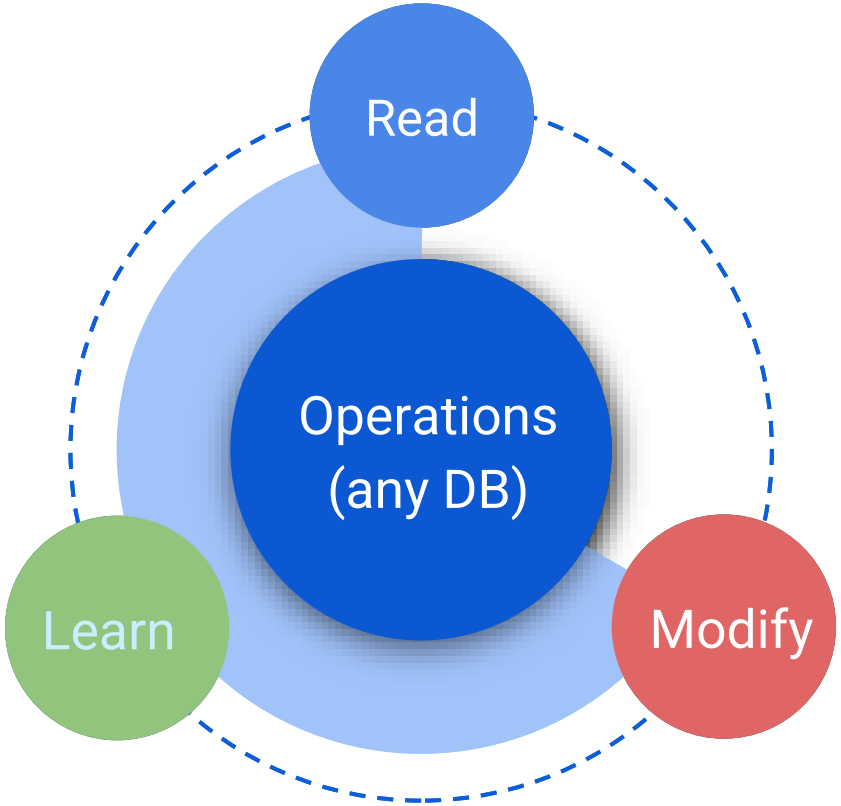
Read Balance
Give money
Update Balance

vs

Read Balance
Update Balance
Give money

Goals of Databases

Platform to store, manage data



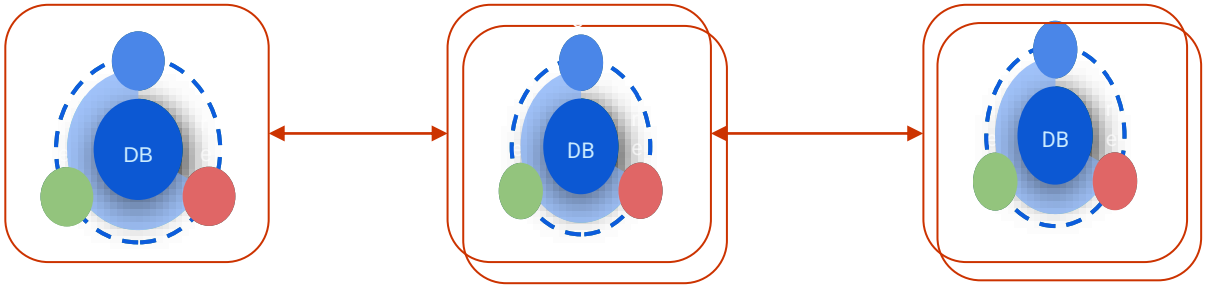
Supporting

- Scale
- Speed
- Stability
- Evolution
- Reliability
- Cost efficiency

Tune for custom system (a la Lego blocks)

Connect one/many DBs for custom system

Goals of Data systems



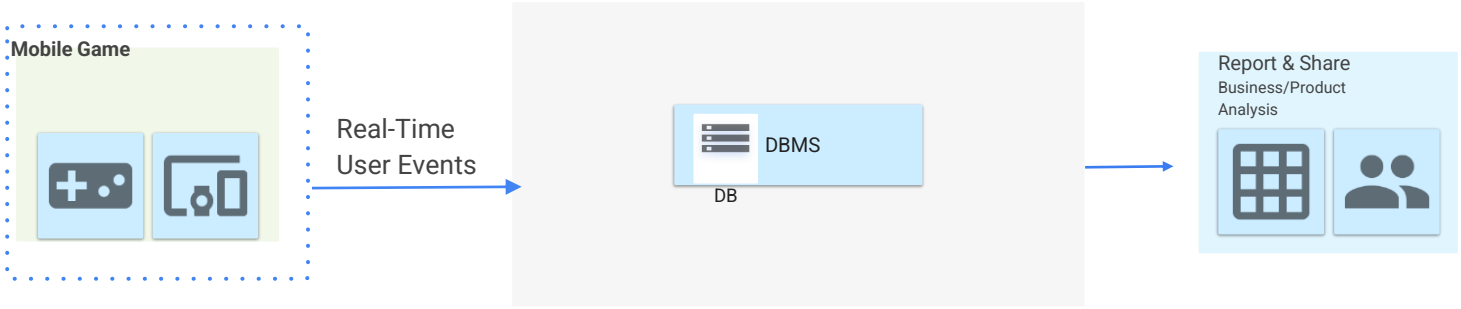
Store current data (e.g., lot of reads)

Optimize historical data (e.g., logs)

Run batch Workloads (e.g. training)

When to build a custom data system?

How?
Example: Game App DB - v0



Q1: 1000 users/sec?
Q2: Offline?
Q3: Support v1, v1' versions?

App designer

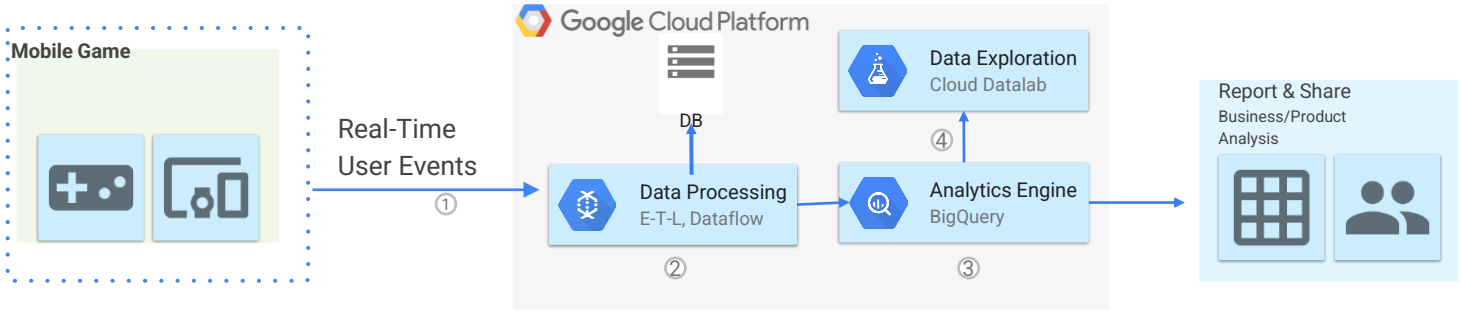
Q7: How to model/evolve game data?
Q8: How to scale to millions of users?
Q9: When machines die, restore game state gracefully?

Systems designer

Q4: Which user cohorts?
Q5: Next features to build? Experiments to run?
Q6: Predict ads demand?

Product/Biz designer

How?
Example: Game App DB – v1 Cloud



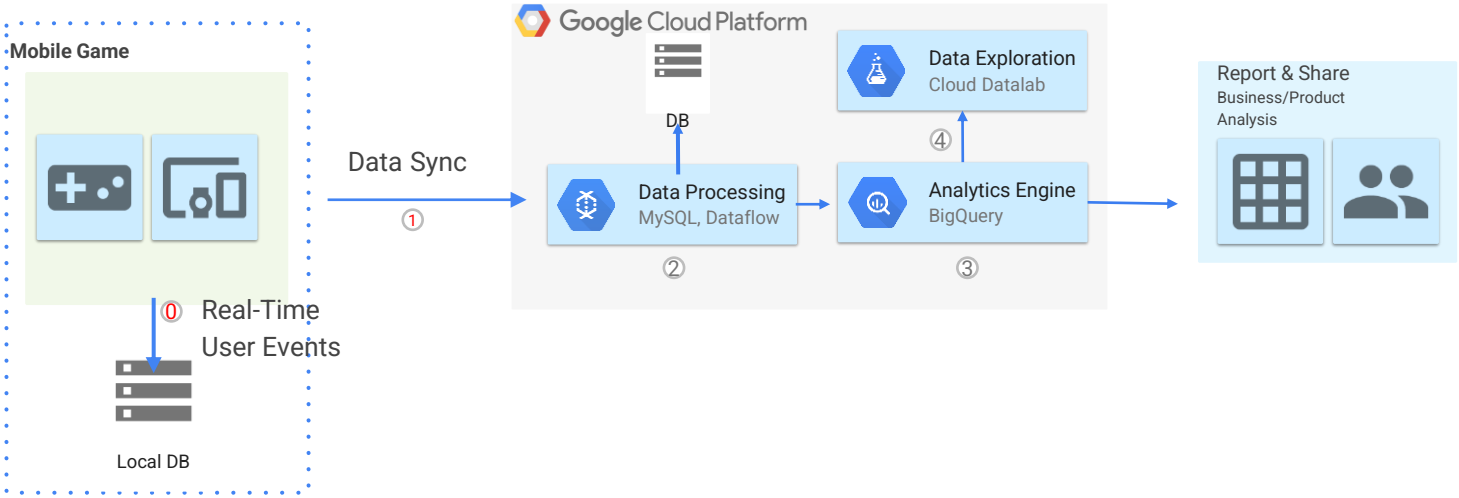
1 Log user actions

2 Store in DB, after Extract-Transform-Load

3 Run queries in a peta scale analytics system

4 Visualize query results

How?
Example: Game App DB v2 Cloud + Local



0 Log user actions In local DB

1 Data sync to cloud

2 Store in DB, after ETL

3 Run queries in a petabyte scale analytics system

4 Visualize query results

Case Study: Data Management at X

Post tweet

- A user can publish a new message to their followers (4.6k requests/sec on average, over 12k requests/sec at peak).

Home timeline

- A user can view tweets posted by the people they follow (300k requests/sec).

Challenge

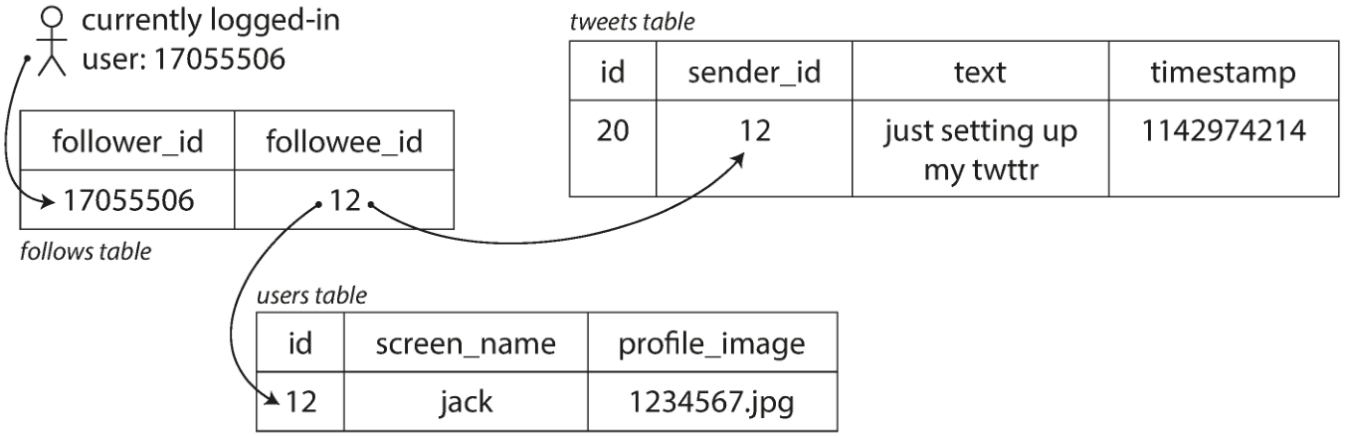
- not due to tweet volume
- but due to fan-out
  - each user follows many people, and each user is followed by many people.

- Alternative ways to implementing these two operations?

Data Management at

Solution 1

- Posting a tweet simply inserts the new tweet into a global collection of tweets.
- When a user requests their home timeline, look up all the people they follow, find all the tweets for each of those users, and merge them (sorted by time).

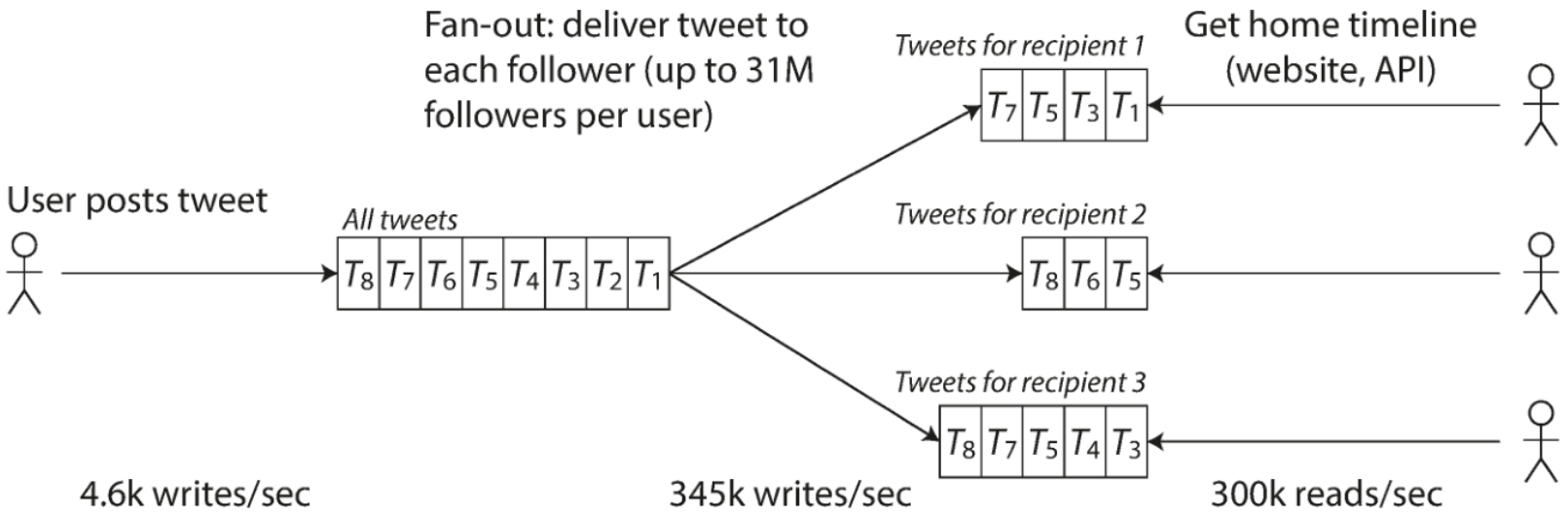



```
SELECT tweets.*, users.* FROM tweets
  JOIN users   ON tweets.sender_id    = users.id
  JOIN follows ON follows.followee_id = users.id
  WHERE follows.follower_id = current_user
```

Data Management at

Solution 2

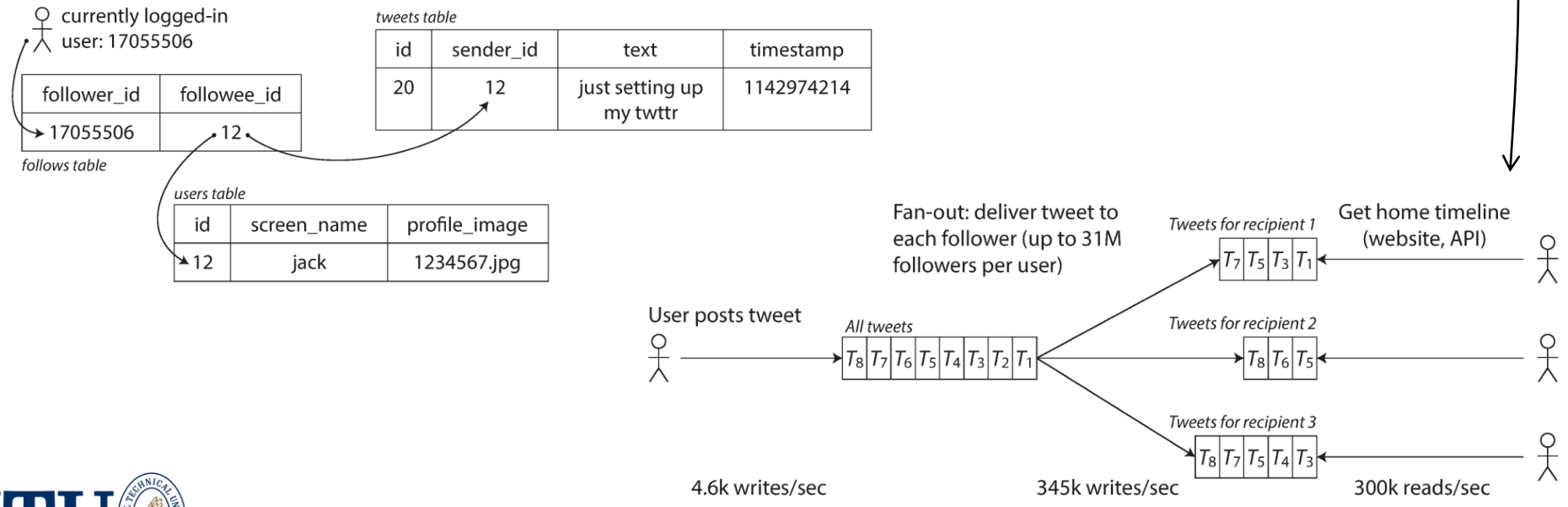
- Maintain a cache for each user's home timeline—like a mailbox of tweets for each recipient user.
- When a user posts a tweet, look up all the people who follow that user, and insert the new tweet into each of their home timeline caches.
- The request to read the home timeline is then cheap, because its result has been computed ahead of time.



Data Management at

Solution 3 (Hybrid)

- Most users' tweets continue to be fanned out to home timelines at the time when they are posted,
- but a small number of users with a very large number of followers (i.e., celebrities) are excepted from this fan-out. Tweets from any celebrities that a user may follow are fetched separately and merged with that user's home timeline when it is read, like in approach 1.



Course Summary

We'll learn…

- Query Execution Algorithms
- Query Optimization
- Transaction Processing
- Indexing for Efficient Data Access
- Distributed Databases
- Parallelism in Databases
- Stream Processing & Real-time Data
- Data Lakehouse Architectures
- NoSQL & NewSQL Systems
- Vector Databases

Course Logistics

Syllabus is available online on Ninova.

Assessment

- Paper review presentations: 10%
- Term Project: 40%
- Mini Exams (Quizzes – 5 of them – 10% each): 50%

Projects

You may choose a project type of:

- Survey paper
- Benchmarking paper
- Research paper

Example topics are posted in syllabus.
Select from this list or propose your own.

Goal:

- **Write a paper that is publishable in a journal in the field.**

Course Logistics

DELIVERABLES, EXAMS, PRESENTATIONS	
Weeks	Deliverable
Week 3	• Project Proposal
Week 4	• Quiz 1
Week 6	• Quiz 2
Week 7	• Paper Review Report • Paper Review Presentation
Week 9	• Quiz 3
Week 10	• Progress Report
Week 11	• Quiz 4
Week 13	• Quiz 5
Week 14	• Term Paper • Project Presentation

Course Logistics

Reference Readings:

- Ramez Elmasri and Shamkant B Navathe. Fundamentals of Database Systems. Hoboken, New Jersey: Pearson, 7th Ed., 2017.
- Kleppmann, M. (2025). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media, Inc.
- Silberschatz, Abraham, Henry F. Korth, and Sudarshan, S. Database System Concepts., 7th ed., New York: McGraw-Hill, 2019.
- Bailis, Peter, Hellerstein, Joseph M., and Stonebraker, Michael. Readings in Database Systems, 5th Edition, ebook available at http://www.redbook.io/, 2015.
- Perkins, L., Redmond, E., & Wilson, J. (2018). Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. Pragmatic Bookshelf.
- Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). "The Case for Learned Index Structures." Proceedings of the 2018 ACM International Conference on Management of Data (SIGMOD '18). [Started the "Learned Indexing" field, arguing that B-Trees can be replaced by neural networks]
- Marcus, R., et al. (2019). "Neo: A Learned Query Optimizer." Proceedings of the VLDB Endowment, 12(11). [Predicting better execution plans and replacing traditional cost models]
- Kipf, Andreas, et al. "Learned Cardinalities: Estimating Correlated Joins with Deep Learning." Proceedings of CIDR 2019.
- Malkov, Y. A., & Yashunin, D. A. (2020). "Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs." IEEE Transactions on Pattern Analysis and Machine Intelligence. [HNSW, the underlying indexing algorithm for almost all modern vector databases (Milvus, Pinecone, Weaviate)].
- Johnson, J., Douze, M., & Jégou, H. (2021). "Billion-scale similarity search with GPUs." IEEE Transactions on Big Data. [Faiss, a library standard for efficient similarity search, crucial for RAG (Retrieval-Augmented Generation) architectures]
- Armbrust, M., et al. (2021). "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics." Proceedings of CIDR 2021. [Introducing the Lakehouse architecture (Delta Lake/Iceberg), moving beyond the Hadoop/MapReduce paradigms].
- Verbitski, A., et al. (2017). "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases." Proceedings of SIGMOD '17. [Essential for understanding "Separation of Compute and Storage", the standard design pattern for modern cloud databases]
- Lamb, A., et al. (2012). "The Vertica Analytic Database: C-Store 7 Years Later." Proceedings of the VLDB Endowment, 5(12). [Understanding Columnar Stores (C-Store), which underpins modern OLAP systems (ClickHouse, Snowflake, BigQuery].
- Akidau, T., et al. (2015). "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing." Proceedings of the VLDB Endowment, 8(12). [Modern streaming concepts like "Watermarks" and "Windowing," essential for IoT Data Streams]
- Carbone, P., et al. (2015). "Apache Flink: Stream and Batch Processing in a Single Engine." Bulletin of the IEEE Computer Society Technical Committee on Data Engineering. [Moving from "Batch-only" (MapReduce) to "Stream-first" processing].

Why?

Why data systems?

What about data structures from algorithms class?

Spreadsheets?

Text files?

Drawbacks of using file systems to store data

- Data redundancy and inconsistency
  - Multiple file formats, duplication of information in different files
- Difficulty in accessing data
  - Need to write a new program to carry out each new task
- Data isolation
  - Inter-relating data stored in multiple files and formats
- Integrity problems
  - Integrity constraints (e.g., account balance > 0) become “buried” in program code rather than being stated explicitly
  - Hard to add new constraints or change existing ones

Drawbacks of using file systems to store data (Cont.)

- Atomicity of updates
  - Failures may leave database in an inconsistent state with partial updates carried out
  - Example: Transfer of funds from one account to another should either complete or not happen at all
- Concurrent access by multiple users
  - Concurrent access needed for performance
  - Uncontrolled concurrent accesses can lead to inconsistencies
    - Example: Two people reading a balance (say 100) and updating it by withdrawing money (say 50 each) at the same time
- Security problems
  - Hard to provide user access to some, but not all, data

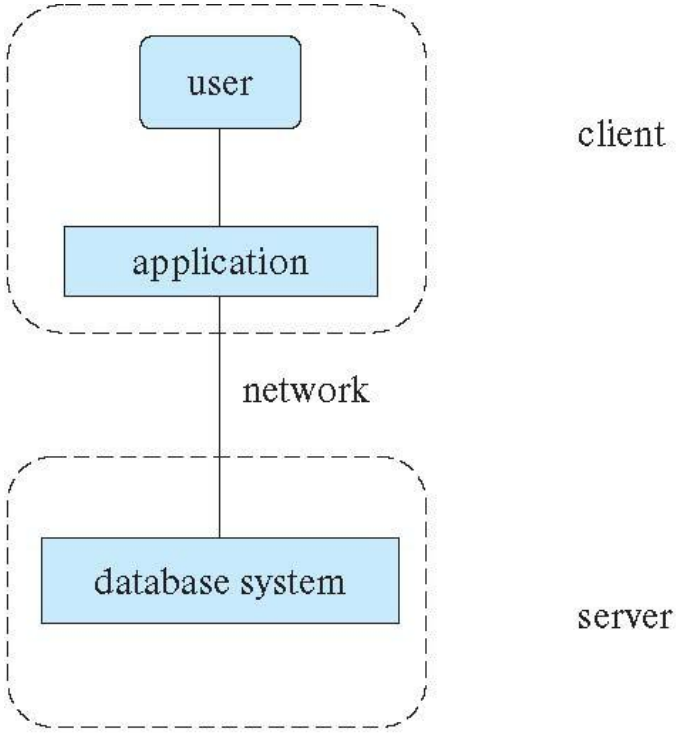
Database systems offer solutions to all the above problems

Database Architectures

- Database systems structure
  - Centralized databases
    - One to a few cores, shared memory
  - Client-server,
    - One server machine executes work on behalf of multiple client machines.
    - Two tier, three tier, …, N-tier architectures

Two-tier Architecture

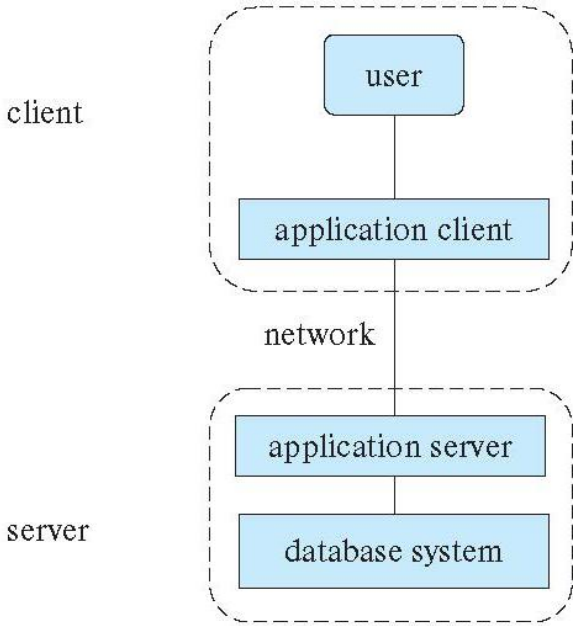
- Database applications are usually partitioned into two or three parts.
  - Two-tier architecture -- the application resides at the client machine, where it invokes database system functionality at the server machine



(a) Two-tier architecture

Three-tier Architecture

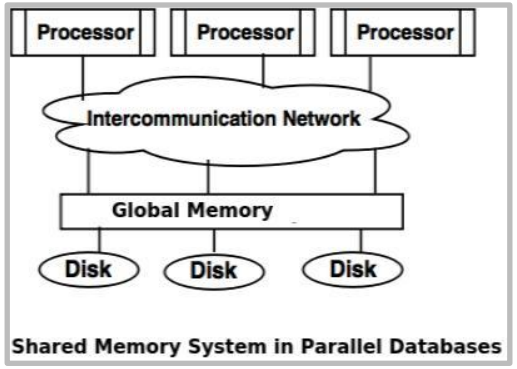
- Database applications are usually partitioned into two or three parts.
  - Three-tier architecture -- the client machine acts as a front end and does not contain any direct database calls.
    - The client end communicates with an application server, usually through a forms interface.
    - The application server in turn communicates with a database system to access data.

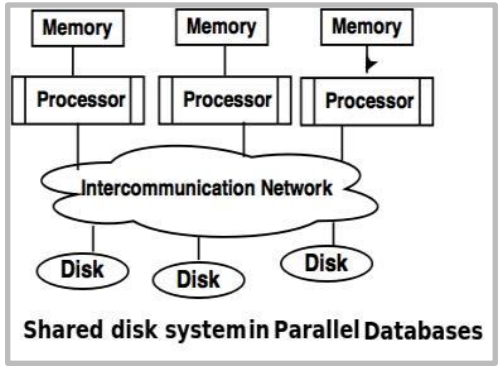


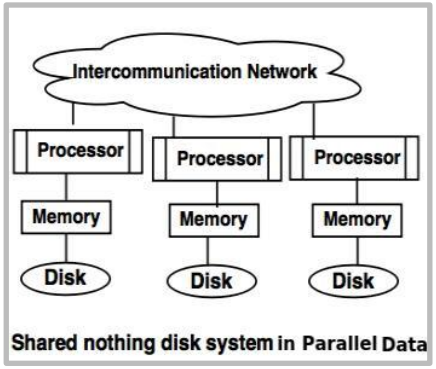
(b) Three-tier architecture

Database Architectures – Cont'd.

- Database systems structure
  - Parallel databases (multiple cores)
    - Shared memory
    - Shared disk
    - Shared nothing

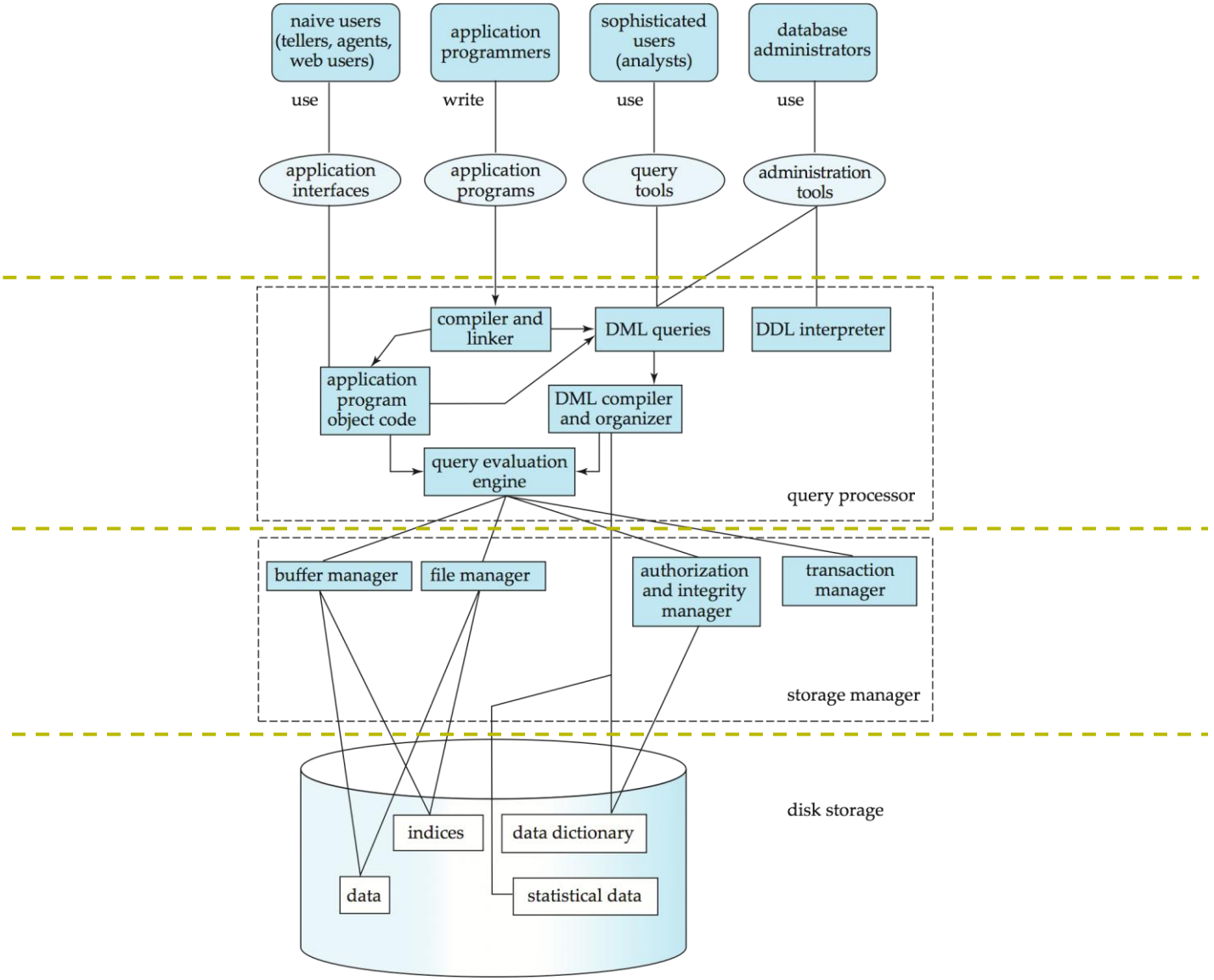






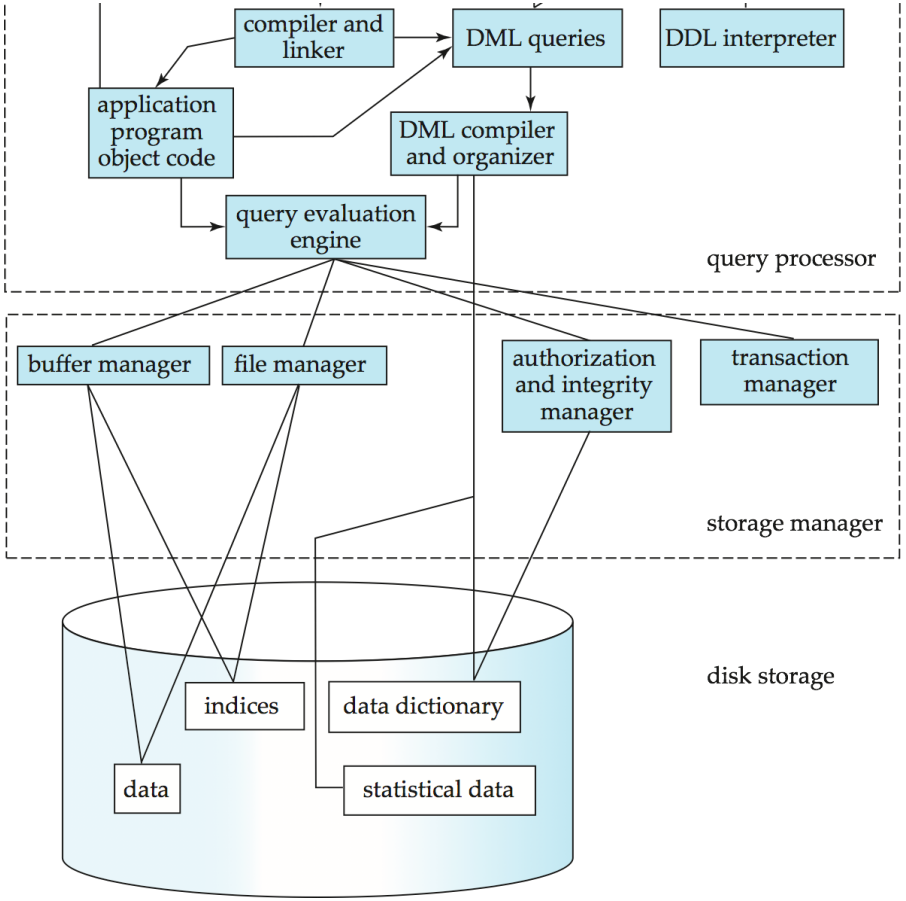
  - Distributed databases
    - Geographical distribution
    - Schema/data heterogeneity

Database System Internals



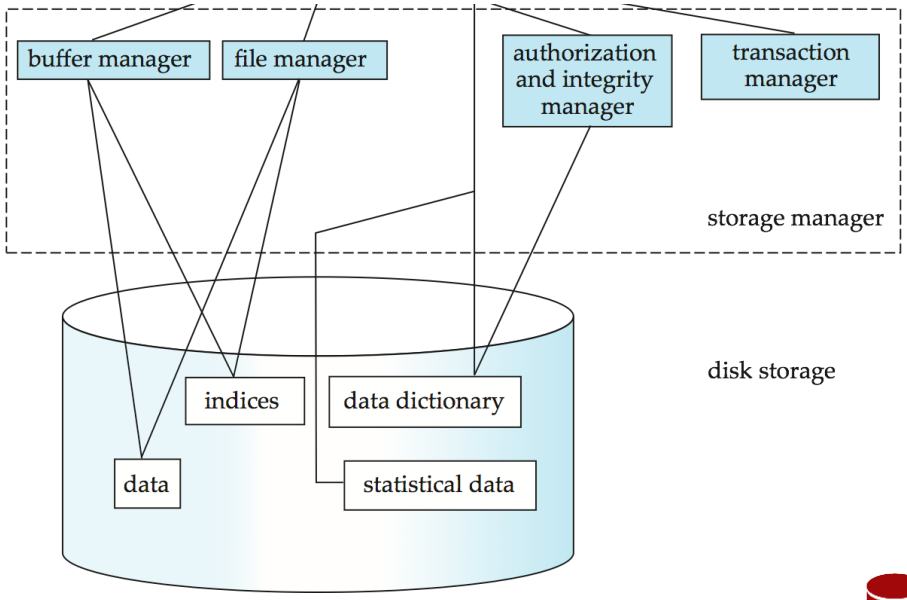
Database Engine

- A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- The functional components of a database system can be divided into
  - The storage manager,
  - The query processor component,
  - The transaction management component.



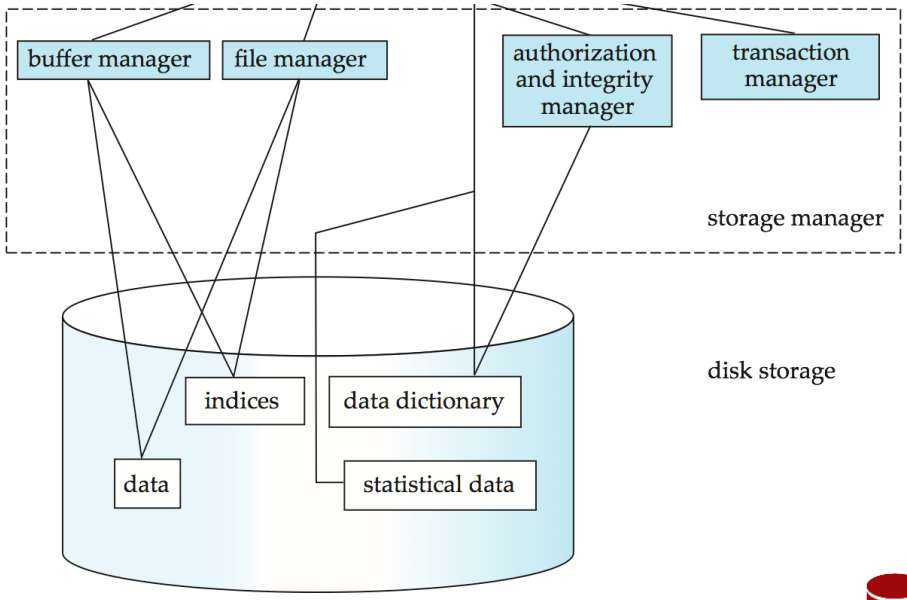
Storage Manager

- A program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
  - Interaction with the OS file manager
  - Efficient storing, retrieving and updating of data
- The storage manager components include:
  - Authorization and integrity manager
  - Transaction manager
  - File manager
  - Buffer manager



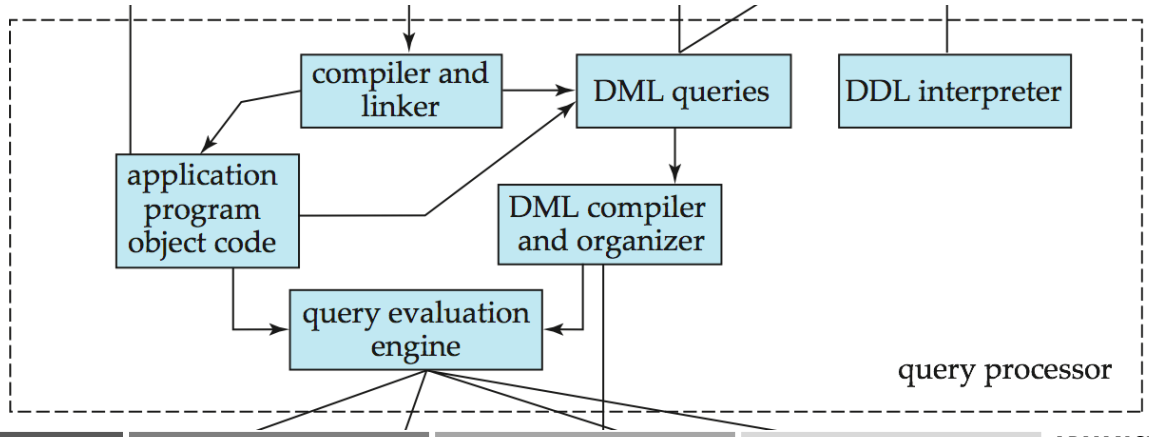
Storage Manager (Cont.)

- The storage manager implements several data structures as part of the physical system implementation:
  - Data files -- store the database itself
  - Data dictionary -- stores metadata about the structure of the database, in particular the schema of the database.
  - Indices -- can provide fast access to data items. A database index provides pointers to those data items that hold a particular value.



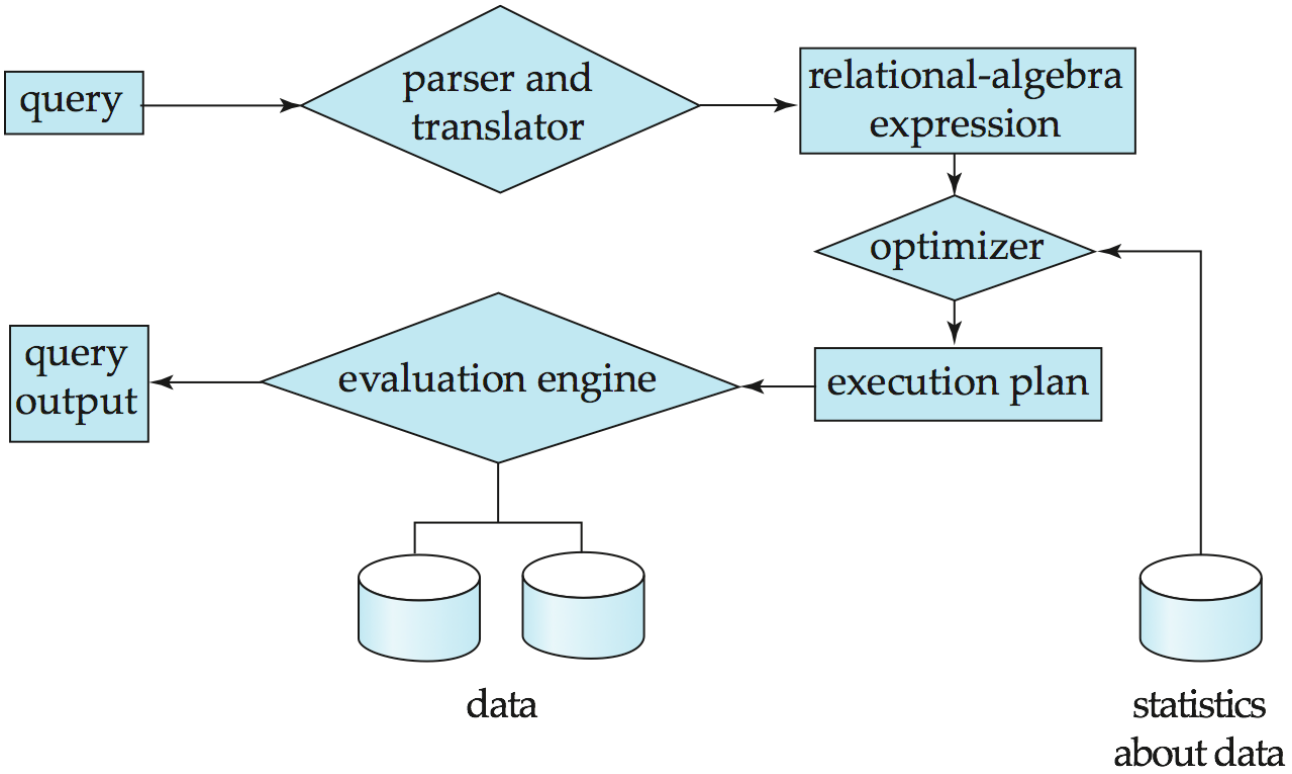
Query Processor

- The query processor components include:
  - DDL interpreter -- interprets DDL statements and records the definitions in the data dictionary.
  - DML compiler -- translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
    - The DML compiler performs query optimization; that is, it picks the lowest cost evaluation plan from among the various alternatives.
  - Query evaluation engine -- executes low-level instructions generated by the DML compiler.



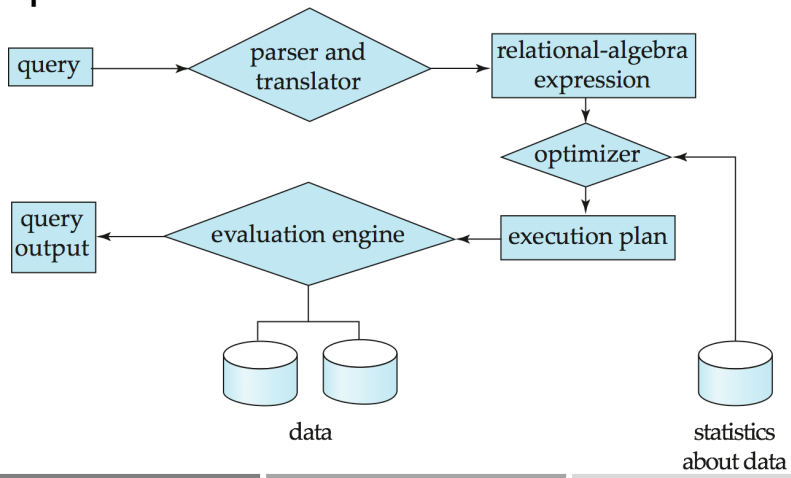
Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Query Processing (Cont.)

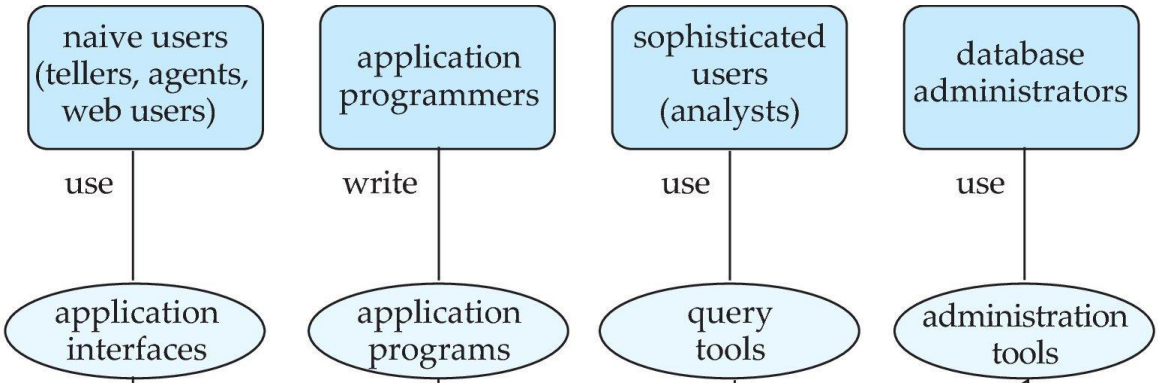
- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation
- Cost difference between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
  - Depends critically on statistical information about relations which the database must maintain / ML Models are getting traction as well
  - Need to estimate statistics for intermediate results to compute cost of complex expressions



Database Users

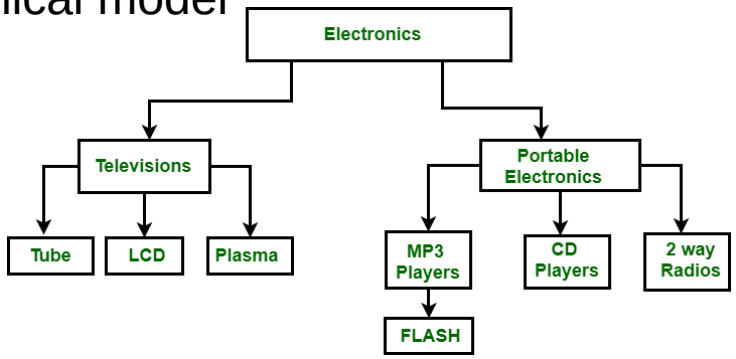
There are four different types of database-system users

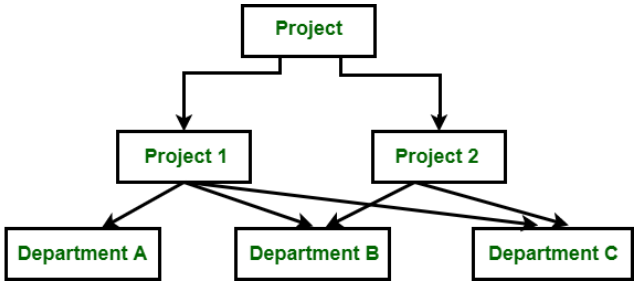
- Naive users -- unsophisticated users who interact with the system by invoking one of the application programs that have been written previously.
- Application programmers -- are computer professionals who write application programs.
- Sophisticated users -- interact with the system without writing programs
  - using a database query language or by
  - using tools such as data analysis software.
- Database Administrators – see the next slide



Data Models

- A collection of tools for describing
  - Data
  - Data relationships
  - Data semantics
  - Data constraints
- Relational model
  - Entity-Relationship data model (mainly for database design)
- Object-based data models (Object-oriented and Object-relational)
- Semistructured data model (XML)
- Other legacy models not in use:
  - Network model
  - Hierarchical model





Relational Model

‘Modeling’ the CMS

Logical Schema

Students(sid: *string*, name: *string*, gpa: *float*)

Courses(cid: *string*, cname: *string*, credits: *int*)

Enrolled(sid: *string,* cid*: string,* grade*: string*)

sid	Name	Gpa
101	Bob	3.2
123	Mary	3.8

Students

Corresponding *keys*

sid	cid	Grade
123	564	A

Enrolled

cid	cname	credits
564	564-2	4
308	417	2

Courses

Set algebra (reminder)

List: [1, 1, 2, 3]

Set: {1, 2, 3}

Multiset: {1, 1, 2, 3}

A **multiset** is an unordered list (or: a set with multiple duplicate instances allowed)

UNIONs

Set: {1, 2, 3} U { 2 } = { 1, 2, 3 }

Multiset: {1, 1, 2, 3} U { 2 } = { 1, 1, 2, 2, 3 }

Tables

Product

PName	Price	Manuf
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

A **relation** or **table** is a multiset of tuples having the attributes specified by the schema

Let's break this definition down

Tables

Product

PName	Price	Manuf
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

An **attribute** (or **column**) is a typed data entry present in each tuple in the relation

*Attributes must have an **atomic** type in standard SQL, i.e. not a list, set, etc.*

Tables

Product

PName	Price	Manuf
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

A **tuple** or **row** is a single entry in the table having the attributes specified by the schema

*Also referred to sometimes as a **Record***

Tables

Product

PName	Price	Manuf
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

The number of tuples is the **cardinality** of the relation

The number of attributes is the **arity** of the relation

Table Schemas

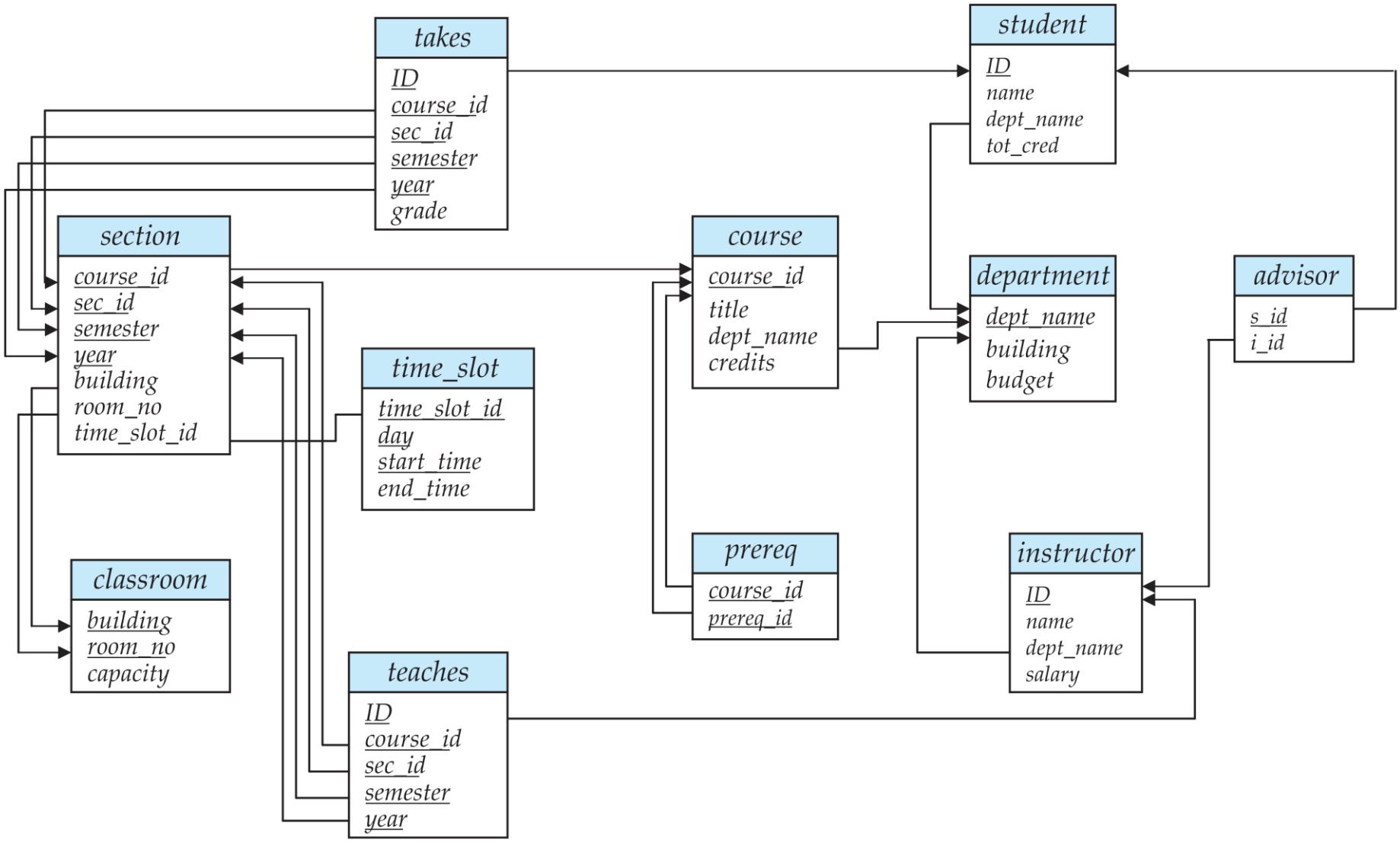
The **schema** of a table is the table name, its attributes, and their types:

Product(Pname: *string*, Price: *float*, Category: *string*, Manufacturer: *string*)

A **key** is an attribute or a set of attributes whose values are unique; we underline a key.

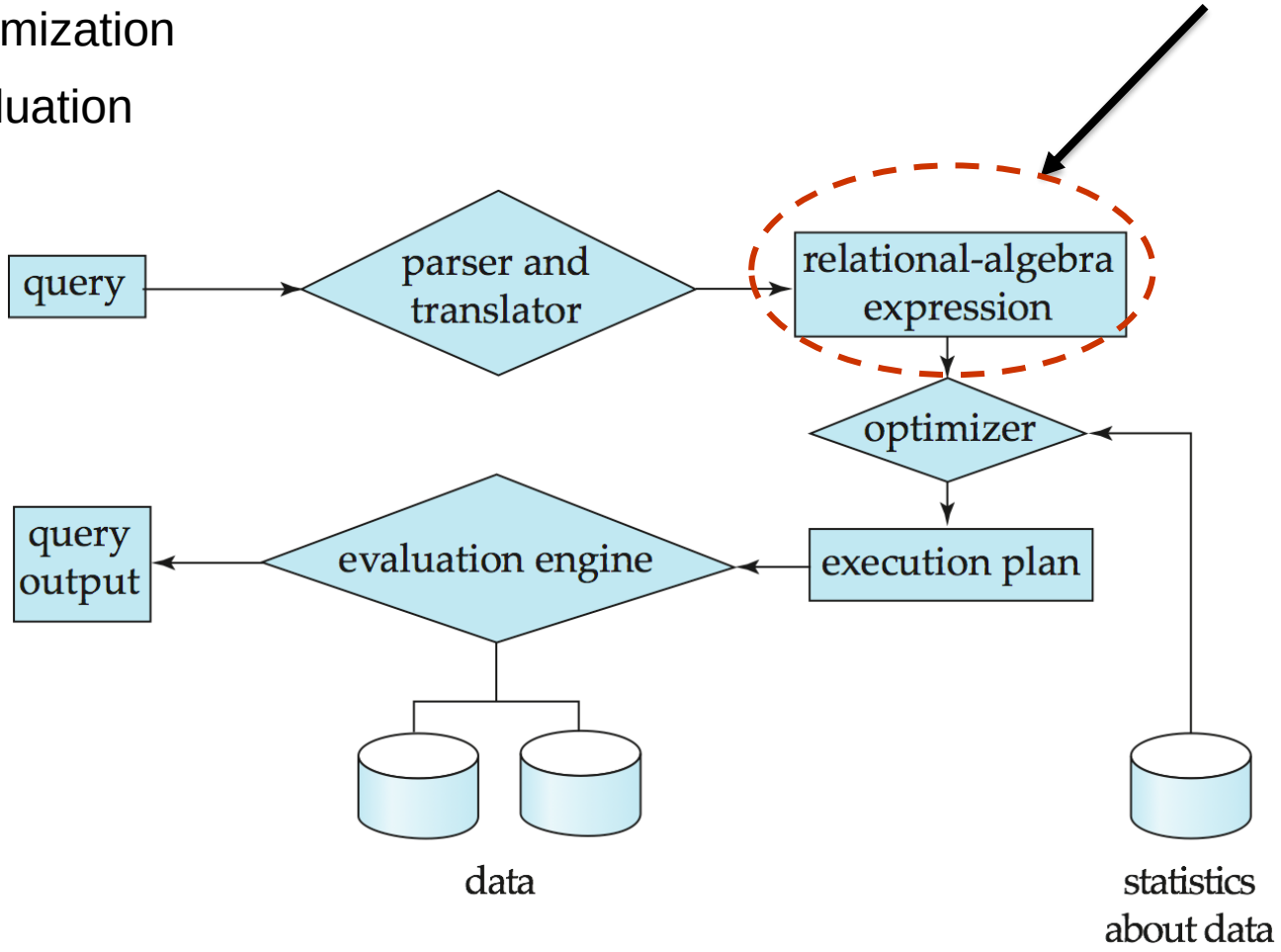
Product(<u>Pname</u>: *string*, Price: *float*, Category: *string*, <u>Manufacturer</u>: *string*)

Schema Diagram for University Database



Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Relational Algebra

- A procedural language consisting of a set of operations
  - take one or two relations as input
  - produce a new relation as a result.

- Six basic operators
  - select: σ
  - project: Π
  - union: $\cup$
  - set difference: –
  - cartesian product: x
  - rename: ρ

Select Operation

- The **select** operation selects tuples that satisfy a given predicate.
- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Example: select those tuples of the *instructor* relation where the instructor is in the "Physics" department.
  - Query

$$\sigma_{dept\_name = \text{“Physics”}}(instructor)$$

  - Result

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Select Operation (Cont.)

- We allow comparisons using

  $=, \neq, >, \geq, <, \leq$

  in the selection predicate.
- We can combine several predicates into a larger predicate by using the connectives:

  $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)
- Example: Find the instructors in Physics with a salary greater $90,000, we write:

$$\sigma_{dept\_name = \text{“Physics”} \wedge salary > 90,000}(instructor)$$

- Besides, select predicate may include comparisons between two attributes.
  - Example, find all departments whose name is the same as their building name:
  - $\sigma_{dept\_name = building}(department)$

Project Operation (Cont.)

- Example: eliminate the *dept_name* attribute of *instructor*
- Query*:*

$$\Pi_{ID,\ name,\ salary}\ (instructor)$$

- Result:

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

ID	*name*	*salary*
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Composition of Relational Operations

- The result of a relational-algebra operation is relation.
- Therefore, relational-algebra operations can be composed together into a **relational-algebra expression**.
- Consider the query -- Find the names of all instructors in the Physics department.

$$\Pi_{name}(\sigma_{dept\_name = \text{“Physics”}}(instructor))$$

- Instead of giving the name of a relation as the argument of the projection operation, we give an expression that evaluates to a relation.

Cartesian-Product Operation

Student

S_id	Name	Class	Age
1	Ali	5	25
2	Veli	10	30
3	Ayse	8	35

Course

C_id	C_name
11	Database Sys
21	Operating Sys

Student X Course

S_id	Name	Class	Age	C_id	C_name
1	Ali	5	25	11	Database Sys
1	Ali	5	25	21	Operating Sys
2	Veli	10	30	11	Database Sys
2	Veli	10	30	21	Operating Sys
3	Ayse	8	35	11	Database Sys
3	Ayse	8	35	21	Operating Sys

The *instructor* x *teaches* table

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

x

ID	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

instructor.ID	*name*	*dept_name*	*salary*	*teaches.ID*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2018
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2017
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2018
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2017
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2018
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

Join Operation

- The table corresponding to:

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches))$$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

Join Operation (Cont.)

- The **join** operation allows us to combine a select operation and a Cartesian-Product operation into a single operation.
- Consider relations *r* (with schema *R*) and *s* (with schema *S*)
- Let "theta" be a predicate on attributes in the schema R "union" S. The join operation r $\bowtie_\theta$ s is defined as follows:

$$r \bowtie_\theta s = \sigma_\theta(r \times s)$$

- Thus

$$\sigma_{instructor.id = teaches.id}(instructor \times teaches)$$

- Can equivalently be written as

$$instructor \bowtie_{Instructor.id = teaches.id} teaches$$

Join Example

$$\text{Student} \bowtie_{\text{Student.Std} = \text{Course.Class}} (\text{Course})$$

Student

S_id	Name	Std	Age
1	Ali	5	25
2	Veli	10	30
3	Ayse	8	35

Course

Class	C_name
10	Database Sys
5	Operating Sys

$$\text{Student} \bowtie_{\text{Student.Std} = \text{Course.Class}} (\text{Course})$$

S_id	Name	Std	Age	Class	C_name
1	Ali	5	25	5	Operating Sys
2	Veli	10	30	10	Database Sys

Union Operation

- Result of:

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2009}(section)) \cup \Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2010}(section))$$

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Set-Intersection Operation

- The set-intersection operation allows us to find tuples that are in both the input relations.
  - Notation: $r \cap s$
- Assume:
  - r, s have the *same arity*
  - attributes of r and s are compatible
- Example: Find the set of all courses taught in both the Fall 2009 and the Spring 2010 semesters.

$$\Pi_{course\_id}\,(\sigma_{semester=\text{“Fall”}\ \wedge\ year=2009}\,(section)) \cap \Pi_{course\_id}\,(\sigma_{semester=\text{“Spring”}\ \wedge\ year=2010}\,(section))$$

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-101

Set Difference Operation

- The set-difference operation allows us to find tuples that are in one relation but are not in another.
- Notation $r - s$
- Set differences must be taken between **compatible** relations.
  - r and s must have the same arity
  - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\Pi_{course\_id}(\sigma_{semester=\text{“Fall”} \wedge year=2009}(section)) - \Pi_{course\_id}(\sigma_{semester=\text{“Spring”} \wedge year=2010}(section))$$

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-347
PHY-101

Equivalent Queries

- There is more than one way to write a query in relational algebra.
- Example: Find instructors in the Physics department with salary greater than 90,000
  - Query 1

$$\sigma_{dept\_name = \text{“Physics”} \wedge salary > 90{,}000}(instructor)$$

  - Query 2

$$\sigma_{dept\_name = \text{“Physics”}}(\sigma_{salary > 90.000}(instructor))$$

- The two queries are not identical;
- They are, however, equivalent –
  - they give the same result on any database.

ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Glimpse Ahead: Efficient Implementations of Operators

- $\sigma_{(age >= 30 \text{ AND } age <= 35)}(Employees)$
  - Method 1: scan the file, test each employee
  - Method 2: use an index on **age**
  - Which one is better ? Depends a lot…
- Employees $\bowtie$ Relatives
  - Iterate over Employees, then over Relatives
  - Iterate over Relatives, then over Employees
  - Sort Employees, Relatives, do “merge-join”
  - “hash-join”
  - Etc.

Glimpse Ahead: Optimizations

Product (pid, name, price, category, maker-cid)
Purchase (buyer-ssn, seller-ssn, store, pid)
Person(ssn, name, phone number, city)

- Which is better:

$$\sigma_{price>100}(Product) \bowtie (Purchase \bowtie \sigma_{city=sea}Person)$$

$$(\sigma_{price>100}(Product) \bowtie Purchase) \bowtie \sigma_{city=sea}Person$$

- Depends ! This is the optimizer's job…

RA has Limitations

- Cannot compute “transitive closure”

Name1	Name2	Relationship
Fred	Mary	Father
Mary	Joe	Cousin
Mary	Bill	Spouse
Nancy	Lou	Sister

- Find all direct and indirect relatives of Fred
- Cannot express in RA !!! Need to write a program

STRUCTURED QUERY LANGUAGE

SQL Introduction

- SQL is a standard language for querying and manipulating data
- Many standards out there:
  ANSI SQL, SQL92 (a.k.a. SQL2), SQL99 (a.k.a. SQL3), ….

SQL stands for
Structured
Query
Language

SQL is a…

- Data Definition Language (DDL)
  - Define relational schemata
  - Create/alter/delete tables and their attributes
- Data Manipulation Language (DML)
  - Query one or more tables
  - Insert/delete/modify tuples in tables

DDL: Create Table Construct

- An SQL relation is defined using the **create table** command:

  create table *r*

  (A_1 D_1, A_2 D_2, ..., A_n D_n,
  (integrity-constraint$_1$),
  ...,
  (integrity-constraint$_k$))

  - *r* is the name of the relation
  - each A_i is an attribute name in the schema of relation *r*
  - D_i is the data type of values in the domain of attribute A_i

- Example:

  create table *instructor* (
  ID **char**(5),
  name **varchar**(20)**,**
  dept_name **varchar**(20),
  salary **numeric**(8,2))

SQL Query (DML)

- Basic form (there are many many more bells and whistles)

```
SELECT <attributes>
FROM   <one or more relations>
WHERE  <conditions>
```

Call this a **SFW** query.

Simple SQL Query: Selection

Selection is the operation of filtering a relation's tuples on some condition

```
SELECT *
FROM   Product
WHERE  Category = 'Gadgets'
```

PName	Price	Category	Manuf
Gizmo	$19.99	Gadgets	GWorks
Powergizmo	$29.99	Gadgets	GWorks
SingleTouch	$149.99	Photography	Canon
MultiTouch	$203.99	Household	Hitachi

PName	Price	Category	Manuf
Gizmo	$19.99	Gadgets	GWorks
Powergizmo	$29.99	Gadgets	GWorks

Keys and Foreign Keys

Company

CName	StockPrice	Country
GizmoWorks	25	USA
Canon	65	Japan
Hitachi	15	Japan

What is a foreign key vs. a key here?

Product

PName	Price	Category	Manufacturer
Gizmo	$19.99	Gadgets	GizmoWorks
Powergizmo	$29.99	Gadgets	GizmoWorks
SingleTouch	$149.99	Photography	Canon
MultiTouch	$203.99	Household	Hitachi

Joins

Ex: Find all products that are **under $200 and manufactured in Japan**; return their **names and prices**.

Product(PName, Price, Category, Manufacturer)

Company(CName, StockPrice, Country)

Several equivalent ways to write a basic join in SQL:

```
SELECT PName, Price
FROM   Product
JOIN   Company
ON     Manufacturer = Cname
WHERE  Price <= 200
   AND Country='Japan'
```

```
SELECT PName, Price
FROM   Product, Company
WHERE  Manufacturer = CName
   AND Price <= 200
   AND Country='Japan'
```

Joins

Product

PName	Price	Category	Manufacturer
Gizmo	$19	Gadgets	GizmoWorks
Powergizmo	$29	Gadgets	GizmoWorks
SingleTouch	$149	Photography	Canon
MultiTouch	$203	Household	Hitachi

Company

CName	Stock Price	Country
GizmoWorks	25	USA
Canon	65	Japan
Hitachi	15	Japan

```
SELECT PName, Price
FROM   Product, Company
WHERE Manufacturer = CName
      AND Country='Japan'
      AND Price <= 200
```

PName	Price
SingleTouch	$149

Semantics of JOINs

SELECT $x_1.a_1$, $x_1.a_2$, …, $x_n.a_k$
FROM R_1 AS x_1, R_2 AS x_2, …, R_n AS x_n
WHERE Conditions(x_1,…, x_n)

Answer = {}
for x_1 **in** R_1 **do**
  for x_2 **in** R_2 **do**
    …..
      for x_n **in** R_n **do**
        if Conditions(x_1,…, x_n)
          then Answer = Answer ∪ {($x_1.a_1$, $x_1.a_2$, …, $x_n.a_k$)}
return Answer

Note:
This is a *multiset* union

Semantics of JOINs (2 tables)

```
SELECT R.A
FROM   R, S
WHERE  R.A = S.B
```

1. Take **cross product**:

Recall: Cross product (A X B) is the set of all unique tuples in A,B
Ex: {a,b,c} X {1,2}
= {(a,1), (a,2), (b,1), (b,2), (c,1), (c,2)}

2. Apply **selections / conditions**:

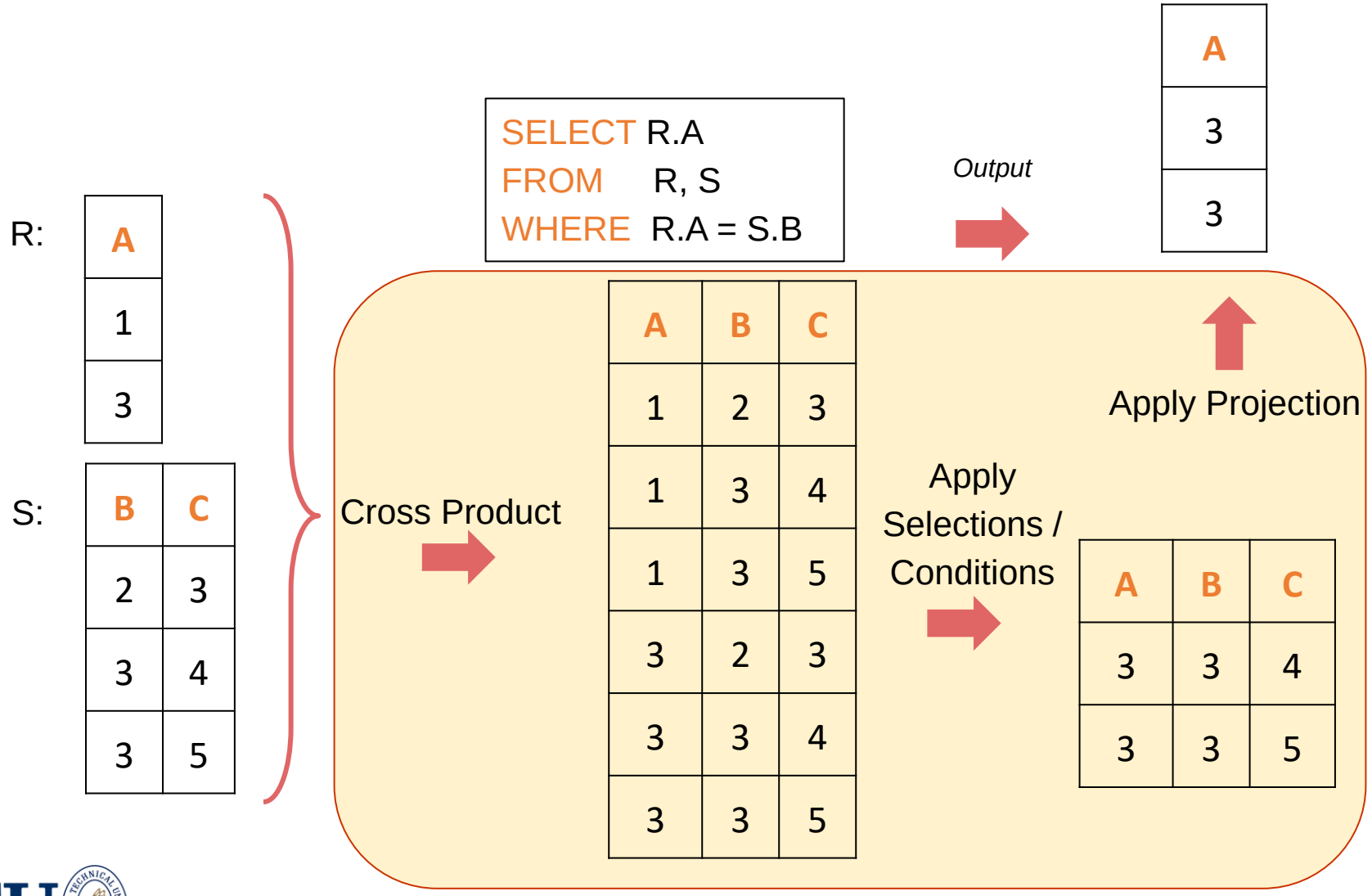
= Filtering!

3. Apply **projections** to get final output:

= Returning only *some* attributes

Remembering this order is critical to understanding the output of certain queries (see later on…)

An example of SQL semantics



Note: we say “semantics” not “execution order”

- The preceding slides show *what a join means*
- Not actually how the DBMS executes it under the covers

Set Operators & Nested Queries

UNION

sales2005

person	amount
Joe	1000
Alex	2000
Bob	5000

sales2006

person	amount
Joe	2000
Alex	2000
Zach	35000

```
SELECT * FROM sales2005
UNION
SELECT * FROM sales2006;
```

person	amount
Joe	1000
Alex	2000
Bob	5000
Joe	2000
Zach	35000

Why aren't there duplicates?

By default:
SQL retains set semantics for UNIONs, INTERSECTs!

What if we want duplicates?

UNION ALL

sales2005

person	amount
Joe	1000
Alex	2000
Bob	5000

sales2006

person	amount
Joe	2000
Alex	2000
Zach	35000

```
SELECT * FROM sales2005
UNION ALL
SELECT * FROM sales2006;
```

person	amount
Joe	1000
Joe	2000
Alex	2000
Alex	2000
Bob	5000
Zach	35000

Set Operations – Examples

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*

- Find courses that ran in Fall 2017 or in Spring 2018

(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)
union
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)

- Find courses that ran in Fall 2017 and in Spring 2018

(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)
intersect
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)

- Find courses that ran in Fall 2017 but not in Spring 2018

(**select** *course_id* **from** *section* **where** *sem* = 'Fall' **and** *year* = 2017)
except
(**select** *course_id* **from** *section* **where** *sem* = 'Spring' **and** *year* = 2018)

Nested queries: Sub-queries Return Relations

An example:

Company(<u>name</u>, city)
Product(<u>name</u>, maker)
Purchase(<u>id</u>, product, buyer)

"Find the locations of companies which make products bought by Mickey"

```
SELECT c.city
FROM   Company c
WHERE  c.name  IN (
       SELECT  pr.maker
       FROM    Purchase p, Product pr
       WHERE   p.product = pr.name
           AND  p.buyer = 'Mickey')
```

High-level note on nested queries

- We can do nested queries because SQL is ***compositional:***
  - Everything (inputs / outputs) is represented as multisets- the output of one query can thus be used as the input to another (nesting)!
- This is extremely powerful!

Subqueries Return Relations

You can also use operations of the form:

- s > ALL R
- s < ANY R → (ANY a.k.a SOME)
- EXISTS R

Ex: Product(name, price, category, maker)

```
SELECT name
FROM   Product
WHERE  price > ALL(
            SELECT price
            FROM   Product
            WHERE  maker = 'Gizmo-Works')
```

Find products that are more expensive than all those produced by "Gizmo-Works"

Subqueries Returning Relations

You can also use operations of the form:

- s > ALL R
- s < ANY R
- EXISTS R

Ex: Product(name, price, category, maker)

```
SELECT p1.name
FROM   Product p1
WHERE  p1.maker = 'Gizmo-Works'
    AND EXISTS(
        SELECT p2.name
        FROM   Product p2
        WHERE  p2.maker <> 'Gizmo-Works'
            AND p1.name = p2.name)
```

<> means !=

Find 'copycat' products, i.e. products made by competitors with the same names as products made by "Gizmo-Works"

Definition of “all” Clause

- F <comp> **all** $r \Leftrightarrow \forall\ t \in r$ (F <comp> *t)*

(5 < **all** | 0, 5, 6) = false

(5 < **all** | 6, 10) = true

(5 = **all** | 4, 5) = false

(5 ≠ **all** | 4, 6) = true (since 5 ≠ 4 and 5 ≠ 6)

Set Comparison – “all” Clause

- Find the names of all instructors whose salary is greater than the salary of all instructors in the Biology department.

ID	*name*	*dept_name*	*salary*

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept name = 'Biology');
```

Definition of "some/any"

- F <comp> **some** $r \Leftrightarrow \exists\, t \in r$ such that (F <comp> t)
  Where <comp> can be: $<,\ \leq,\ >,\ =,\ \neq$

(5 < **some** | 0 | 5 | 6 |) = true (read: 5 < some tuple in the relation)

(5 < **some** | 0 | 5 |) = false

(5 = **some** | 0 | 5 |) = true

(5 ≠ **some** | 0 | 5 |) = true (since 0 ≠ 5)

(= **some**) ≡ **in**

Set Comparison – “some/any”

- Find names of instructors with salary greater than that of some (at least one) instructor in the Biology department.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept name = 'Biology';
```

- Same query using > **some** clause

```
select name
from instructor
where salary > some (select salary
                     from instructor
                     where dept name = 'Biology');
```

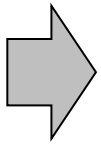
Alternative: “any”

ID	name	dept_name	salary

Nested queries as alternatives to INTERSECT and EXCEPT

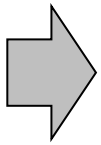
INTERSECT and EXCEPT not in some DBMSs!

```
(SELECT R.A, R.B
 FROM   R)
INTERSECT
(SELECT S.A, S.B
 FROM   S)
```



```
SELECT R.A, R.B
FROM   R
WHERE EXISTS(
      SELECT *
      FROM S
      WHERE R.A=S.A AND R.B=S.B)
```

```
(SELECT R.A, R.B
 FROM   R)
EXCEPT
(SELECT S.A, S.B
 FROM   S)
```



```
SELECT R.A, R.B
FROM   R
WHERE NOT EXISTS(
      SELECT *
      FROM S
      WHERE R.A=S.A AND R.B=S.B)
```

Correlated Queries Using External Vars in Internal Subquery

Movie(title, year, director, length)

```
SELECT DISTINCT title
FROM   Movie AS m
WHERE  year <> ANY(
            SELECT  year
            FROM    Movie
            WHERE   title =  m.title)
```

Find movies whose title appears more than once.

Note the scoping of the variables!

Note also: this can still be expressed as single SFW query…

Complex Correlated Query

Product(name, price, category, maker, year)

```
SELECT DISTINCT  x.name, x.maker
FROM   Product AS x
WHERE  x.price > ALL(
          SELECT y.price
          FROM   Product AS y
          WHERE  x.maker = y.maker
            AND  y.year < 1972)
```

Find products (and their manufacturers) that are more expensive than all products made by the same manufacturer before 1972

Can be very powerful (also much harder to optimize)

Aggregation

```
SELECT AVG(price)
FROM   Product
WHERE  maker = “Toyota”
```

```
SELECT COUNT(*)
FROM   Product
WHERE  year > 1995
```

- SQL supports several **aggregation** operations:
  - SUM, COUNT, MIN, MAX, AVG

Except COUNT, all aggregations apply to a single attribute

Grouping and Aggregation

What GROUPings are possible?

- Type, Size, Color
- Number of holes
- Combination?

Grouping and Aggregation

Purchase(product, date, price, quantity)

```
SELECT   product, SUM(price * quantity) AS TotalSales
FROM     Purchase
WHERE    date > '10/1/2005'
GROUP BY product
```

Find total sales after 10/1/2005 per product.

Let's see what this means…

Grouping and Aggregation

```
SELECT   product,
         SUM(price * quantity) AS TotalSales
FROM     Purchase
WHERE    date > '10/1/2005'
GROUP BY product
```

Semantics of the query:

1. Compute the FROM and WHERE clauses

2. Group by the attributes in the GROUP BY

3. Compute the SELECT clause: grouped attributes and aggregates

1. Compute the FROM and WHERE clauses

```
SELECT   product, SUM(price*quantity) AS
TotalSales
FROM     Purchase
WHERE    date > '10/1/2005'
GROUP BY product
```

FROM

Product	Date	Price	Quantity
Bagel	10/21	1	20
Bagel	10/25	1.50	20
Banana	10/3	0.5	10
Banana	10/10	1	10

2. Group by the attributes in the GROUP BY

```
SELECT   product, SUM(price*quantity) AS
TotalSales
FROM     Purchase
WHERE    date > '10/1/2005'
GROUP BY product
```

Product	Date	Price	Quantity
Bagel	10/21	1	20
Bagel	10/25	1.50	20
Banana	10/3	0.5	10
Banana	10/10	1	10

GROUP BY →

Product	Date	Price	Quantity
Bagel	10/21	1	20
	10/25	1.50	20
Banana	10/3	0.5	10
	10/10	1	10

3. Compute the SELECT clause: grouped attributes and aggregates

```
SELECT   product,
SUM(price*quantity) AS TotalSales
FROM     Purchase
WHERE    date > '10/1/2005'
GROUP BY product
```

Product	Date	Price	Quantity
Bagel	10/21	1	20
	10/25	1.50	20
Banana	10/3	0.5	10
	10/10	1	10

SELECT

Product	TotalSales
Bagel	50
Banana	15

HAVING Clause

```
SELECT   product, SUM(price*quantity)
FROM     Purchase
WHERE    date > '10/1/2005'
GROUP BY product
HAVING   SUM(quantity) > 100
```

Same query as before, except that we consider only products that have more than 100 units of sales

HAVING clauses contains conditions on **groups**

*Whereas WHERE clauses condition on **individual tuples…***

General form of Grouping and Aggregation

```
SELECT      S
FROM        R1,…,Rn
WHERE       C1
GROUP BY    a1,…,ak
HAVING      C2
```

Evaluation steps:

1. Evaluate FROM-WHERE: apply condition C_1 on the attributes in $R_1,\ldots,R_n$
2. GROUP BY the attributes $a_1,\ldots,a_k$
3. Apply condition C_2 to each group (may need to compute aggregates)
4. Compute aggregates in S and return the result

Aggregation (Cont.)

- Attributes in **select** clause outside of aggregate functions must appear in **group by** list
  - /* erroneous query */
    select *dept_name*, *ID*, **avg** (*salary*)
    from *instructor*
    group by *dept_name*;

Group-by v.s. Nested Query

Students(sid, name, gpa)
Enrolled(student_id, cid, grade)

- Find students enrolled in > 5 classes
- Attempt 1: with nested queries

```
SELECT Students.sid, Students.name
FROM   Students
WHERE  COUNT(
    SELECT cid
    FROM   Enrolled
    WHERE  Students.sid = Enrolled.student_id) > 5
```

This is SQL by a novice

Group-by v.s. Nested Query

- Attempt 2: SQL style (with GROUP BY)

```
SELECT   Students.sid, Students.name
FROM     Students, Enrolled
WHERE    Students.sid = Enrolled.student_id
GROUP BY Students.sid, Students.name
HAVING   COUNT(Enrolled.cid) > 5
```

This is SQL by an expert

NULLS in SQL

- Whenever we don’t have a value, we can put a NULL
- Can mean many things:
  - Value does not exist
  - Value exists but is unknown
  - Value not applicable
  - Etc.
- The schema specifies for each attribute if can be null (*nullable* attribute) or not
- How does SQL cope with tables that have NULLs?

Null Values

- *For numerical operations,* NULL -> NULL:
  - If x = NULL then 4*(3-x)/7 is still NULL
- *For boolean operations,* in SQL there are three values:

FALSE	**=**	**0**
UNKNOWN	**=**	**0.5**
TRUE	**=**	**1**

  - If x= NULL then x="Joe" is UNKNOWN

Null Values

- C1 AND C2 = min(C1, C2)
- C1 OR C2 = max(C1, C2)
- NOT C1 = 1 – C1

```
SELECT *
FROM   Person
WHERE age < 25
  AND height > 6 AND weight > 190
```

Won't return e.g. (age=20 height=NULL weight=200)!

Rule in SQL: include only tuples that yield TRUE (1.0)

Null Values

Unexpected behavior:

```
SELECT *
FROM   Person
WHERE  age < 25 OR age >= 25
```

Some Persons are not included !

Null Values

Can test for NULL explicitly:

- x IS NULL
- x IS NOT NULL

```
SELECT *
FROM   Person
WHERE  age < 25 OR age >= 25
   OR age IS NULL
```

Now it includes all Persons!

Null Values and Aggregates

- Total all salaries

  select sum (*salary*)
  from *instructor*

  - Above statement ignores null amounts
  - Result is *null* if there is no non-null amount
- All aggregate operations except **count(*)** ignore tuples with null values on the aggregated attributes
- What if collection has only null values?
  - count returns 0
  - all other aggregates return null

Inner Joins → Lost data?

By default, joins in SQL are **"inner joins":**

```
Product(name, category)
Purchase(prodName, store)
```

```
SELECT Product.name, Purchase.store
FROM   Product
  JOIN Purchase ON Product.name = Purchase.prodName
```

```
SELECT Product.name, Purchase.store
FROM   Product, Purchase
WHERE  Product.name = Purchase.prodName
```

However: Products that never sold (with no Purchase tuple) will be lost!

Outer Joins

- An **outer join** returns tuples from the joined relations that don't have a corresponding tuple in the other relations
  - i.e., If we join relations A and B on a.X = b.X, and there is an entry in A with X=5, but none in B with X=5…
  - A LEFT OUTER JOIN will return a tuple (a, NULL)!
- Left outer joins in SQL:

```
SELECT Product.name, Purchase.store
FROM   Product
  LEFT OUTER JOIN Purchase ON
                Product.name = Purchase.prodName
```

Now we'll get products even if they didn't sell

INNER JOIN:

Product

name	category
Gizmo	gadget
Camera	Photo
OneClick	Photo

Purchase

prodName	store
Gizmo	Wiz
Camera	Ritz
Camera	Wiz

```
SELECT Product.name, Purchase.store
FROM   Product
  INNER JOIN Purchase
                ON Product.name = Purchase.prodName
```

name	store
Gizmo	Wiz
Camera	Ritz
Camera	Wiz

Note: another equivalent way to write an INNER JOIN!

LEFT OUTER JOIN:

Product

name	category
Gizmo	gadget
Camera	Photo
OneClick	Photo

Purchase

prodName	store
Gizmo	Wiz
Camera	Ritz
Camera	Wiz

```
SELECT Product.name, Purchase.store
FROM   Product
  LEFT OUTER JOIN Purchase
                ON Product.name = Purchase.prodName
```

name	store
Gizmo	Wiz
Camera	Ritz
Camera	Wiz
OneClick	NULL

Other Outer Joins

- Left outer join:
  - Include the left tuple even if there's no match
- Right outer join:
  - Include the right tuple even if there's no match
- Full outer join:
  - Include the both left and right tuples even if there's no match

Subqueries in the From Clause

Subqueries in the From Clause

- SQL allows a subquery expression to be used in the **from** clause
- Find the average instructor salaries of those departments where the average salary is greater than $42,000."

```
select dept_name, avg_salary
from ( select dept_name, avg (salary) as avg_salary
        from instructor
        group by dept_name)
where avg_salary > 42000;
```

- Note that we do not need to use the **having** clause
- Another way to write the above query

```
select dept_name, avg_salary
from ( select dept_name, avg (salary)
        from instructor
        group by dept_name)
        as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```

With Clause

- The **with** clause provides a way of defining a temporary relation whose definition is available only to the query in which the **with** clause occurs.
- Find all departments with the maximum budget

```
with max_budget (value) as
        (select max(budget)
         from department)
select department.name
from department, max_budget
where department.budget = max_budget.value;
```

Scalar Subquery

- Scalar subquery is one which is used where a single value is expected
- List all departments along with the number of instructors in each department

```
select dept_name,
        ( select count(*)
          from instructor
          where department.dept_name = instructor.dept_name)
        as num_instructors
from department;
```

- Runtime error if subquery returns more than one result tuple

Backup Slides

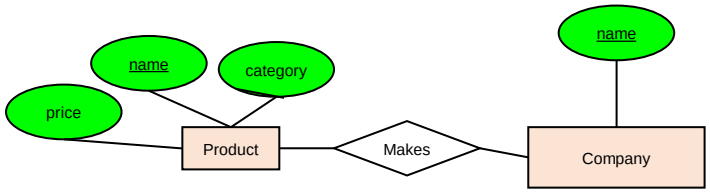
DATABASE DESIGN

Design Alternatives

- In designing a database schema, we must ensure that we avoid two major pitfalls:
  - **Redundancy**: a bad design may result in repeated information.
    - Data inconsistency among the various copies of information
    - Waste of storage space
  - **Incompleteness**: a bad design may make certain aspects of the enterprise difficult or impossible to model.

Design Approaches

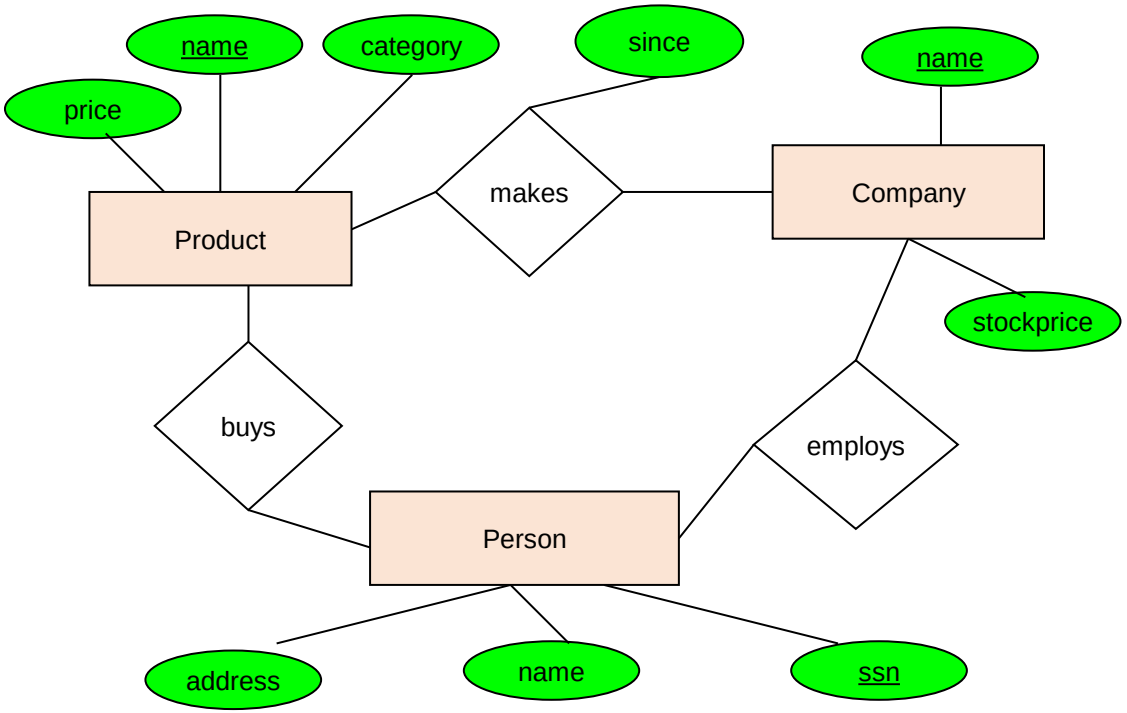
- Entity Relationship Model
  - Models an enterprise as a collection of *entities* and *relationships*
    - Entity: a “thing” or “object” in the enterprise that is distinguishable from other objects
      - Described by a set of *attributes*
    - Relationship: an association among several entities
  - Represented diagrammatically by an *entity-relationship diagram:*



E/R is a *visual syntax* for DB design which is ***precise enough*** for technical points, but ***abstracted enough*** for non-technical people

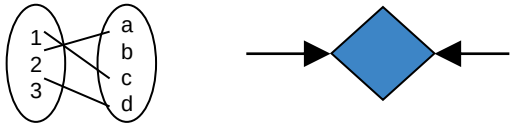
- Normalization Theory:
  - Formalize what designs are bad, and test for them.

An example E/R diagram

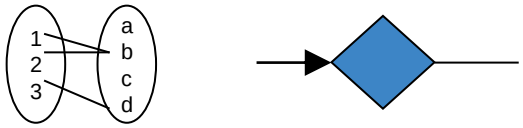


Multiplicity of E/R Relationships

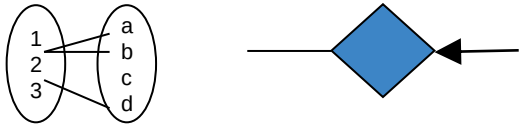
One-to-one:



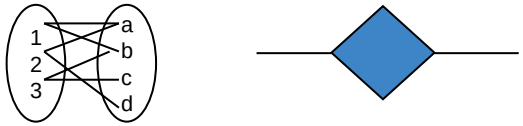
Many-to-one:

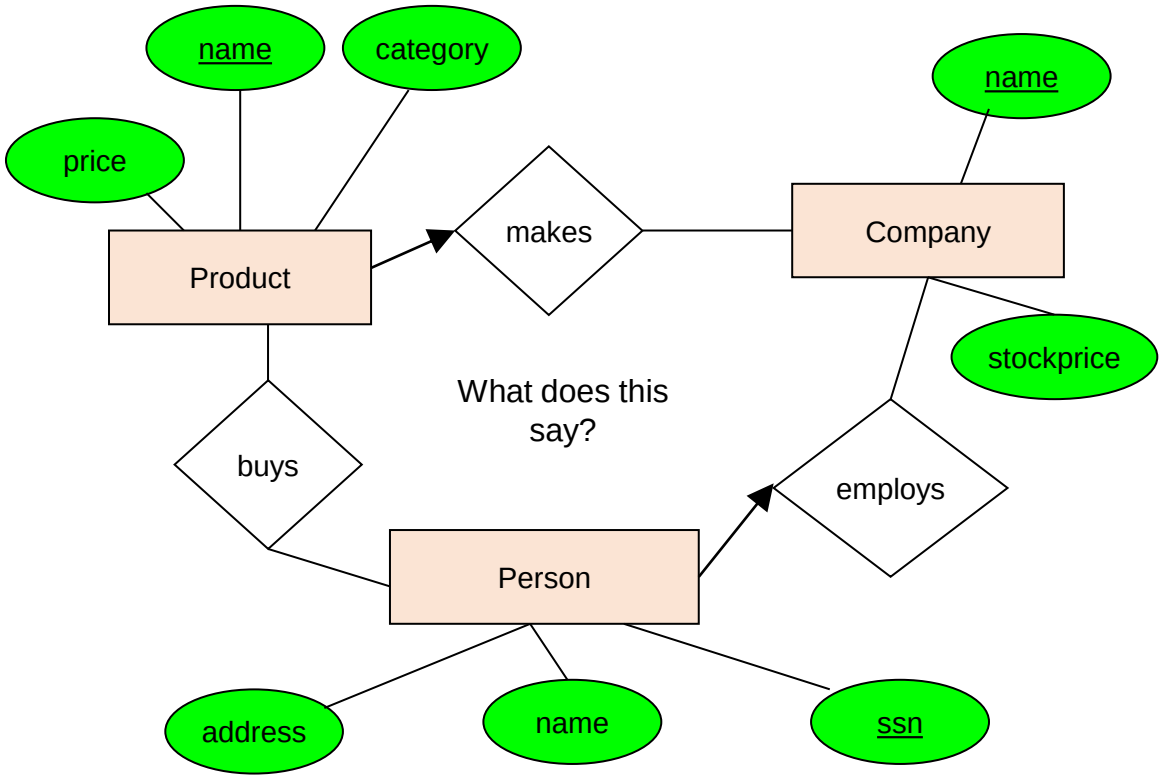


One-to-many:



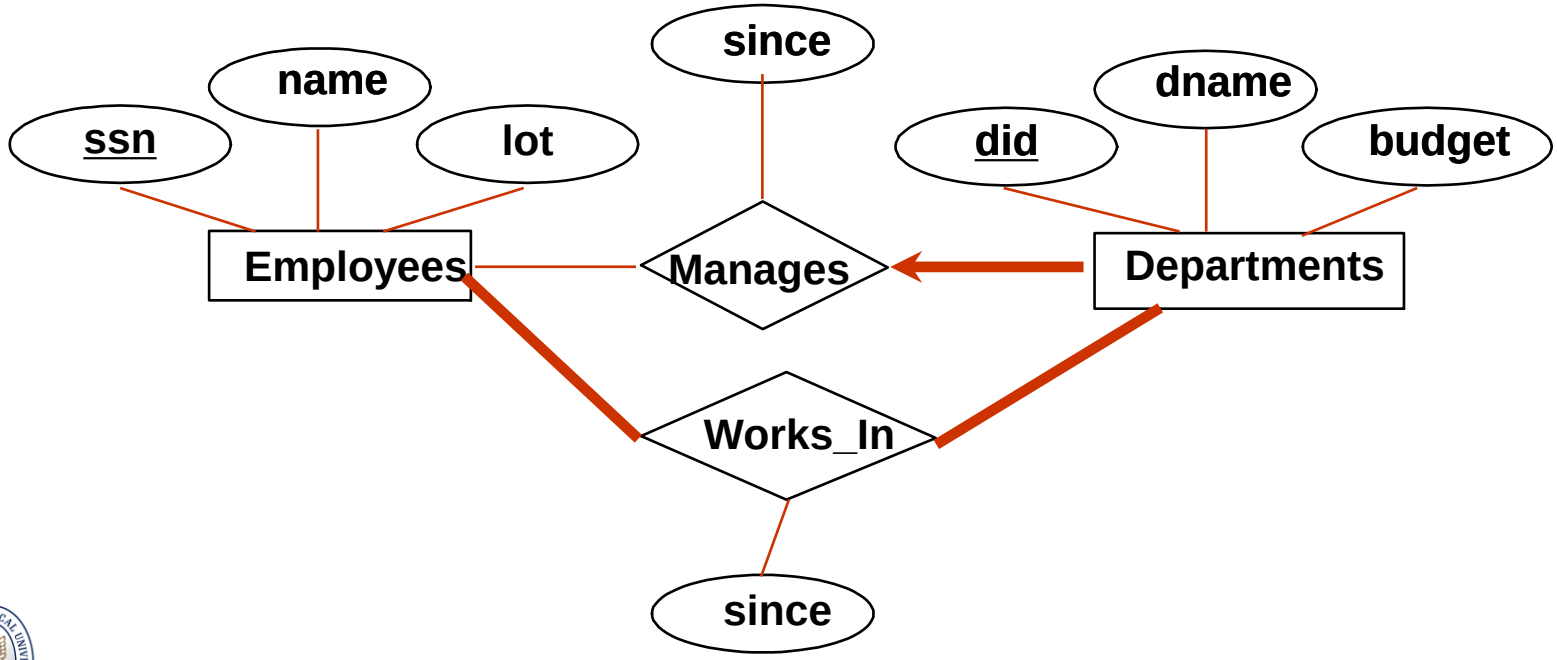
Many-to-many:




name
category
price
Product
makes
name
Company
stockprice
What does this say?
buys
employs
Person
address
name
ssn

Participation Constraints

- Does every department have a manager?
  - If so, this is a *participation constraint*: the participation of Departments in Manages is said to be *total* (vs. *partial*).
    - Every Departments entity must appear in an instance of the Manages relationship.

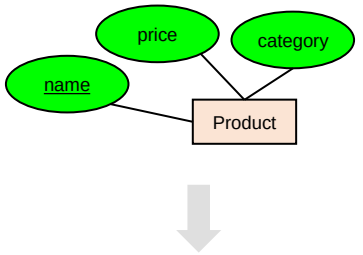


From E/R Diagrams to Relational Schema

- Key concept:
  - Both ***Entity sets*** and ***Relationships*** become relations (tables in RDBMS)

From E/R Diagrams to Relational Schema

- An entity set becomes a relation (multiset of tuples / table)
  - Each tuple is one entity
  - Each tuple is composed of the entity's attributes, and has the same primary key

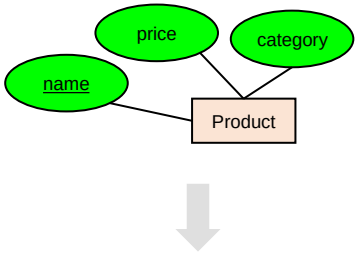


Product

name	price	category
Gizmo1	99.99	Camera
Gizmo2	19.99	Edible

From E/R Diagrams to Relational Schema

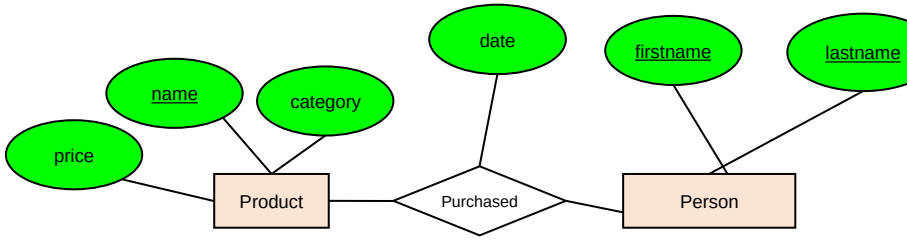
```
CREATE TABLE Product(
  name     CHAR(50) PRIMARY KEY,
  price    DOUBLE,
  category VARCHAR(30)
)
```



Product

name	price	category
Gizmo1	99.99	Camera
Gizmo2	19.99	Edible

From E/R Diagrams to Relational Schema

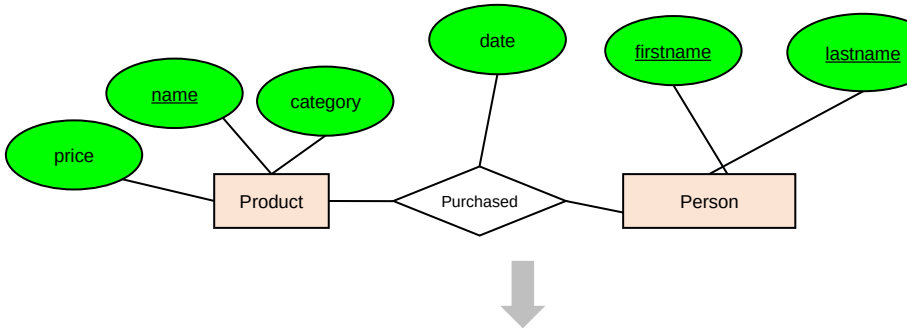


- A relation between entity sets $A_1, \ldots, A_N$ *also* becomes a multiset of tuples / a table
  - Each row/tuple is one relation, i.e. one unique combination of entities $(a_1,\ldots,a_N)$
  - Each row/tuple is
    - composed of the **union of the entity sets' keys**
    - has the entities' primary keys as foreign keys
    - has the union of the entity sets' keys as primary key

Purchased

name	firstname	lastname	date
Gizmo1	Bob	Joe	01/01/15
Gizmo2	Joe	Bob	01/03/15
Gizmo1	JoeBob	Smith	01/05/15

From E/R Diagrams to Relational Schema

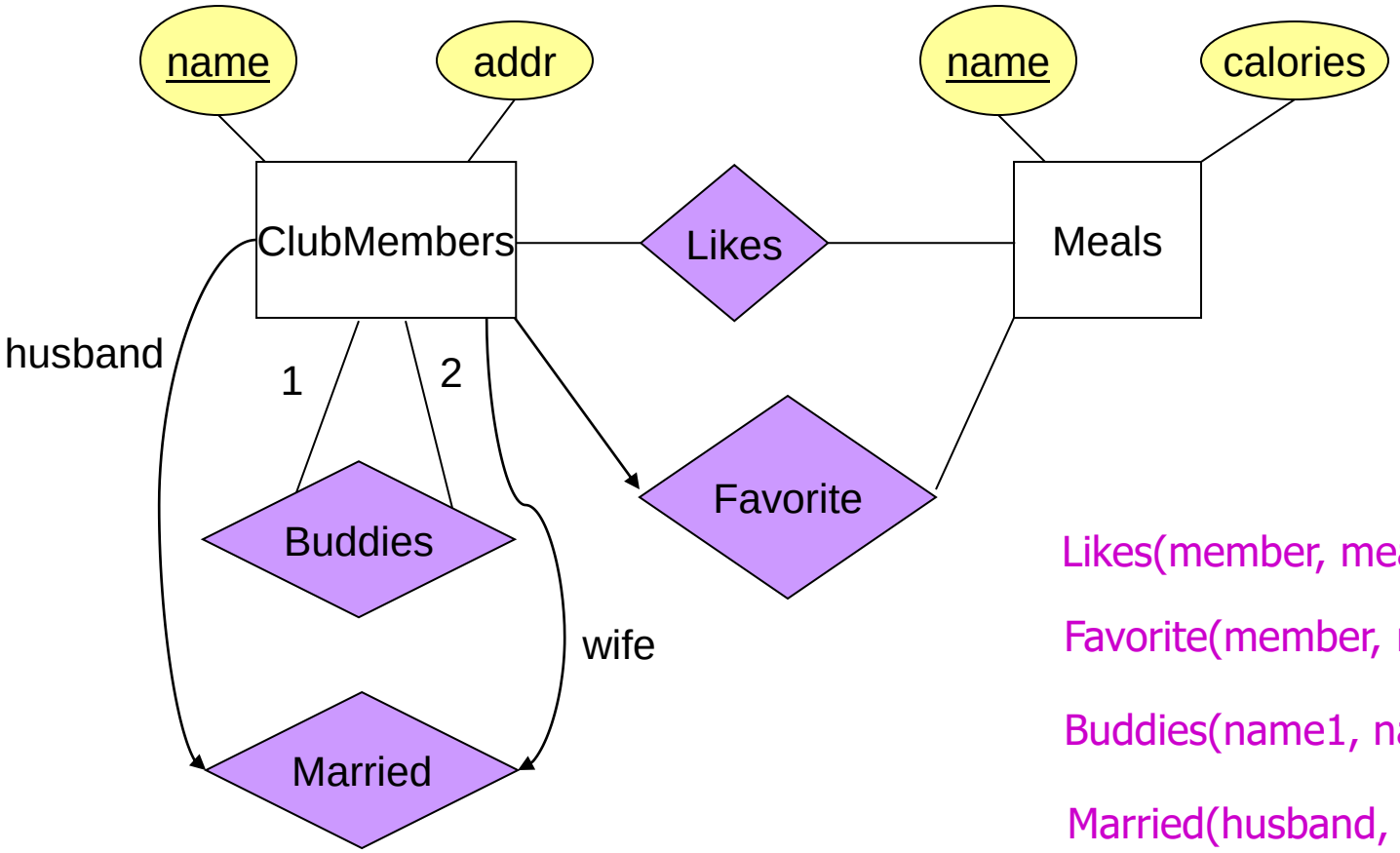



```
CREATE TABLE Purchased(
  name      CHAR(50),
  firstname CHAR(50),
  lastname  CHAR(50),
  date      DATE,
  PRIMARY KEY (name, firstname, lastname),
  FOREIGN KEY (name)
                REFERENCES Product,
  FOREIGN KEY (firstname, lastname)
                REFERENCES Person
)
```

Purchased

name	firstname	lastname	date
Gizmo1	Bob	Joe	01/01/15
Gizmo2	Joe	Bob	01/03/15
Gizmo1	JoeBob	Smith	01/05/15

Relationship -> Relation



Likes(member, meal)

Favorite(member, meal)

Buddies(name1, name2)

Married(husband, wife)

ClubMembers(name, addr, fav_meal)

Meals(name, calories)

Design Theory

Example Enrollment table - "v0"

~375 cs145 students

SID	Class	Room	Time	Lat	Lng
4749732	cs 145	Nvidia Aud	T/R 4:30-6	37.4277° N	122.1742° W
2720942	cs 145	Nvidia Aud	T/R 4:30-6	37.4277° N	122.1742° W
4823984	cs 145	Nvidia Aud	T/R 4:30-6	37.4277° N	122.1742° W
4287594	cs 145	Nvidia Aud	T/R 4:30-6	...	...
2984994	cs 145	Nvidia Aud	T/R 4:30-6	...	...
8472374	cs 145	Nvidia Aud	T/R 4:30-6	...	...
4723663	cs 145	Nvidia Aud	T/R 4:30-6	...	...
2478239	cs 145	Nvidia Aud	T/R 4:30-6	...	...
4763268	cs 145	Nvidia Aud	T/R 4:30-6	...	...
2364532	cs 145	Nvidia Aud	T/R 4:30-6	...	...
2364573	cs 145	Nvidia Aud	T/R 4:30-6	...	...
3476382	cs 145	Nvidia Aud	T/R 4:30-6	...	...
2347623	cs 145	Nvidia Aud	T/R 4:30-6	...	...
...	...	...	...	...	...
2364579	cs 245	Nvidia Aud	T/R 3-4:30	37.4277° N	122.1742° W
3476343	cs 245	Nvidia Aud	T/R 3-4:30	37.4277° N	122.1742° W
2322232	cs 245	Nvidia Aud	T/R 3-4:30	37.4277° N	122.1742° W

~300 cs245 students

Problems
Repeats?
Room/time change?
Deletes?

Example Franken tables

Example Enrollment table - “v1”

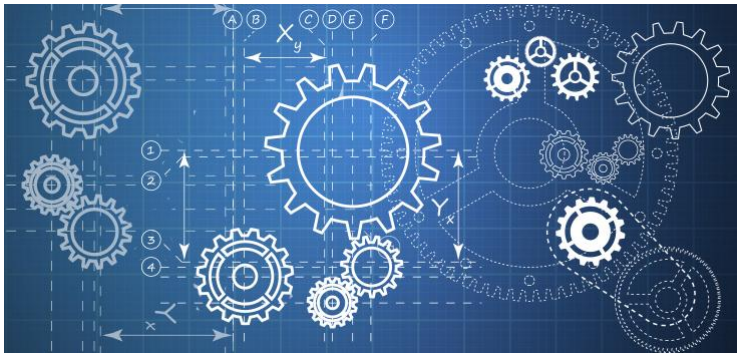
	SID	Class
375 cs145 students	4749732	cs 145
	2720942	cs 145
	4823984	cs 145
	4287594	cs 145
	2984994	cs 145
	8472374	cs 145
	4723663	cs 145
	2478239	cs 145
	4763268	cs 145
	2364532	cs 145
	2364573	cs 145
	3476382	cs 145
	2347623	cs 145
	...	...
300 cs245 students	2364579	cs 245
	3476343	cs 245
	2322232	cs 245

Class	Room	Time
cs 145	Nvidia Aud	T/R 4:30-6
cs 245	Nvidia Aud	T/R 3-4:30
cs 246	Nvidia Aud	M/W 3-4:30

Room	Lat	Lng
Nvidia Aud	37.4277° N	122.1742° W

Design Theory

- Design theory is about how to represent your data to avoid ***anomalies***.
- Simple algorithms for “best practices”



DATA ANOMALIES

Anomalies in the Data

A poorly designed database causes ***anomalies***:

Student	Course	Room
Mary	CS145	B01
Joe	CS145	B01
Sam	CS145	B01
..	..	..

Contains ***redundant*** information!

Anomalies in the Data

A poorly designed database causes ***anomalies***:

Student	Course	Room
Mary	CS145	B01
Joe	CS145	C12
Sam	CS145	B01
..	..	..

If we update the room number for one tuple, we get inconsistent data = an ***update* anomaly**

Anomalies in the Data

A poorly designed database causes ***anomalies***:

Student	Course	Room
..	..	..

If everyone drops the class, we lose what room the class is in! = a ***delete* anomaly**

Anomalies in the Data

A poorly designed database causes ***anomalies***:

…	CS229	C12

→

Student	Course	Room
Mary	CS145	B01
Joe	CS145	B01
Sam	CS145	B01
..	..	..

Similarly, we can't reserve a room without students = an ***insert* anomaly**

Anomalies in the Data

Student	Course
Mary	CS145
Joe	CS145
Sam	CS145
..	..

Course	Room
CS145	B01
CS229	C12

Is this form better?

- Redundancy?
- Update anomaly?
- Delete anomaly?
- Insert anomaly?

Functional Dependencies

Functional Dependency

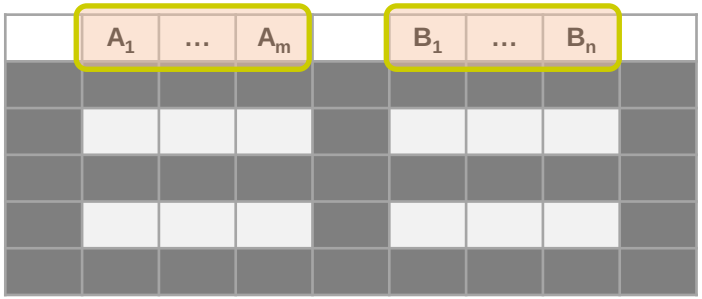
Def: Let A,B be *sets* of attributes
We write A → B or say A ***functionally determines*** B, if for any tuples t_1 and t_2:

$$t_1[A] = t_2[A] \text{ implies } t_1[B] = t_2[B]$$

and we call A → B a **functional dependency**

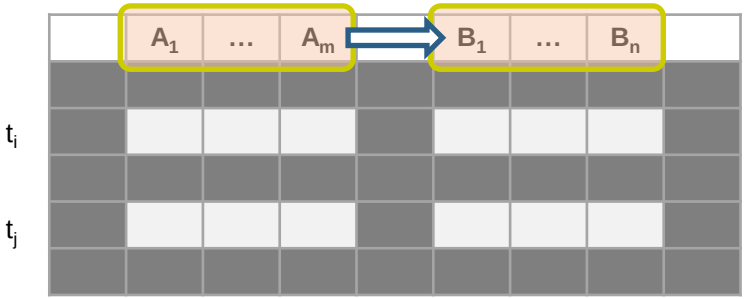
A → B means that
"whenever two tuples agree on A then they agree on B."

A Picture Of FDs



Defn (again):
Given attribute sets **A={A_1,…,A_m}** and **B = {B_1,…B_n}** in **R,**

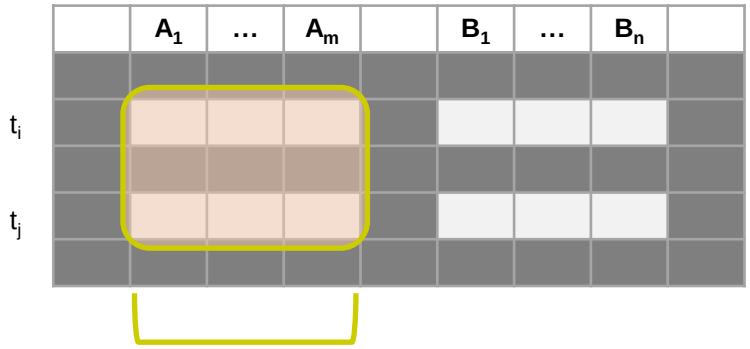
A Picture Of FDs



Defn (again):
Given attribute sets **A={A_1,…,A_m}** and **B = {B_1,…B_n}** in **R,**

The ***functional dependency*** **A→ B on R** holds if for ***any*** t_i,t_j in R:

A Picture Of FDs



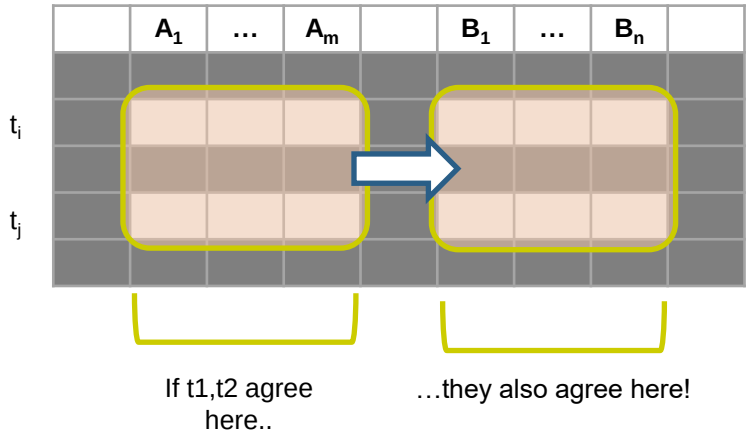
If t1,t2 agree here..

Defn (again):
Given attribute sets **A={A_1,…,A_m}** and **B = {B_1,…B_n}** in **R,**

The ***functional dependency*** **A→ B on R** holds if for ***any*** t_i,t_j in R:

$t_i[A_1] = t_j[A_1]$ AND $t_i[A_2]=t_j[A_2]$ AND … AND $t_i[A_m] = t_j[A_m]$

A Picture Of FDs



Defn (again):
Given attribute sets **A={A_1,…,A_m}** and **B = {B_1,…B_n}** in **R,**

The ***functional dependency*** **A→ B on R** holds if for ***any*** t_i,t_j in R:

if $t_i[A_1] = t_j[A_1]$ AND $t_i[A_2]=t_j[A_2]$ AND … AND $t_i[A_m] = t_j[A_m]$

then $t_i[B_1] = t_j[B_1]$ AND $t_i[B_2]=t_j[B_2]$ AND … AND $t_i[B_n] = t_j[B_n]$

FDs for Relational Schema Design

High-level idea: **why do we care about FDs?**

1. Start with some relational *schema*
2. Find out its *functional dependencies (FDs)*
3. Use these to *design a better schema*

   One which minimizes the possibility of anomalies

Functional Dependencies as Constraints

Student	Course	Room
Mary	CS145	B01
Joe	CS145	B01
Sam	CS145	B01
..	..	..

Note: The FD {Course} -> {Room} ***holds on this instance***

However, cannot *prove* that the FD {Course} -> {Room} is ***part of the schema***

*Recall: an **instance** of a schema is a multiset of tuples conforming to that schema, **i.e. a table***

ACTIVITY

A	B	C	D	E
1	2	4	3	6
3	2	5	1	8
1	4	4	5	7
1	2	4	3	6
3	2	5	1	8

Find at least *three* FDs which are violated on this instance:

{ } → { }
{ } → { }
{ } → { }

“Good” vs. “Bad” FDs

“Good” vs. “Bad” FDs

We can start to develop a notion of **good** vs. **bad** FDs:

EmpID	Name	Phone	Position
E0045	Smith	1234	Clerk
E3542	Mike	9876	Salesrep
E1111	Smith	9876	Salesrep
E9999	Mary	1234	Lawyer

Intuitively:

EmpID -> Name, Phone, Position is *a “good FD”.*

- ***Minimal redundancy, less possibility of anomalies***

“Good” vs. “Bad” FDs

We can start to develop a notion of **good** vs. **bad** FDs:

EmpID	Name	Phone	Position
E0045	Smith	1234	Clerk
E3542	Mike	9876	Salesrep
E1111	Smith	9876	Salesrep
E9999	Mary	1234	Lawyer

Intuitively:

EmpID -> Name, Phone, Position is *a “good FD”.*

But Position -> Phone *is a “bad FD”*

- ***Redundancy!***
- ***Possibility of data anomalies***

Finding functional dependencies

Finding Functional Dependencies

- There can be a very **large number** of FDs…
  - *How to find them all efficiently?*
- We can't necessarily show that any FD will hold **on all instances…**
  - *How to do this?*

We will start with this problem:
Given a set of FDs, F, what other FDs ***must*** hold?

Finding Functional Dependencies

Example:

Products

Name	Color	Category	Dep	Price
Gizmo	Green	Gadget	Toys	49
Widget	Black	Gadget	Toys	59
Gizmo	Green	Whatsit	Garden	99

Provided FDs:

1. {Name} → {Color}
2. {Category} → {Department}
3. {Color, Category} → {Price}

Given the provided FDs, we can see that {Name, Category} → {Price} must also hold on **any instance**…

Which / how many other FDs do?!?

Finding Functional Dependencies

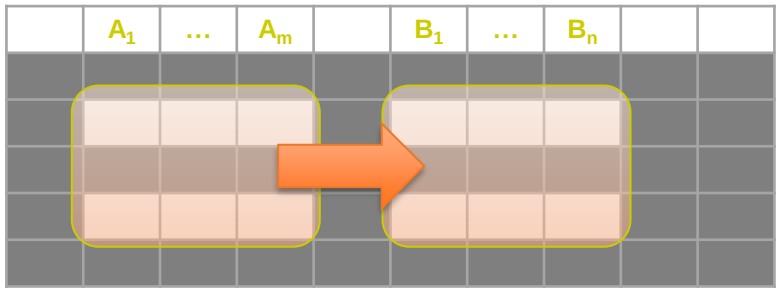
Equivalent to asking: Given a set of FDs, F = {f_1,…f_n}, does an FD g hold?

Inference problem: How do we decide?

Answer: Three simple rules called
Armstrong's Rules.

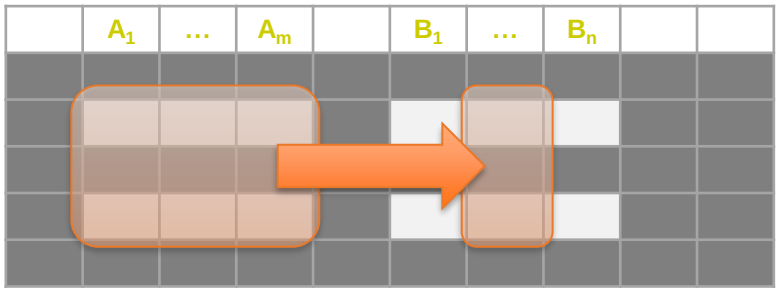
1. **Split/Combine**
2. **Reduction**
3. **Transitivity**

1. Split/Combine



$A_1, \ldots, A_m \rightarrow B_1, \ldots, B_n$

1. Split/Combine

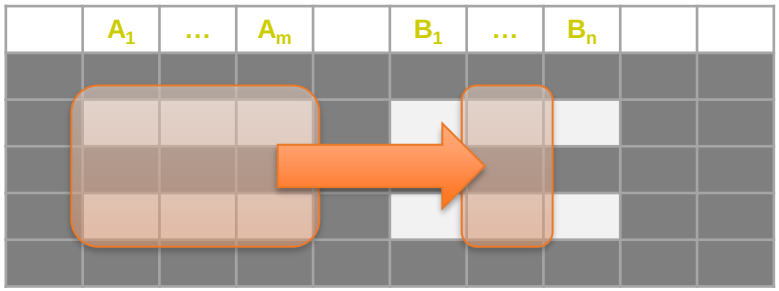


$A_1, \ldots, A_m \rightarrow B_1,\ldots,B_n$

… is equivalent to the following *n* FDs…

$A_1,\ldots,A_m \rightarrow B_i$ for i=1,…,n

1. Split/Combine

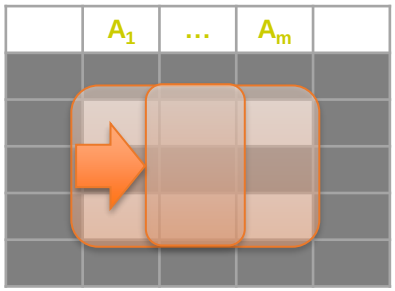


And vice-versa, $A_1,\ldots,A_m \rightarrow B_i$ for $i=1,\ldots,n$

… is equivalent to …

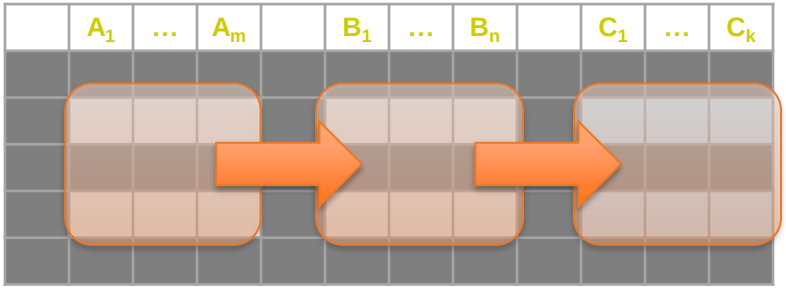
$A_1, \ldots, A_m \rightarrow B_1,\ldots,B_n$

2. Reduction/Trivial



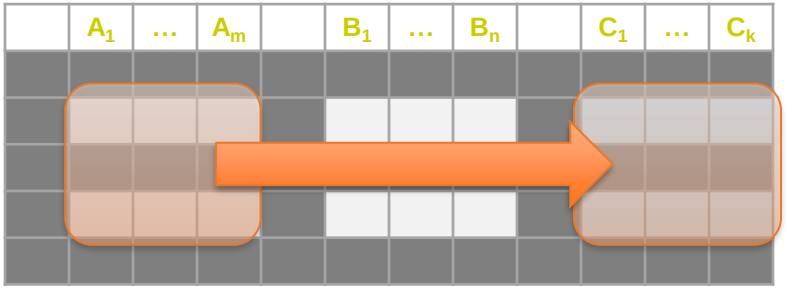
$A_1,\ldots,A_m \rightarrow A_j$ for any $j=1,\ldots,m$

3. Transitive Closure



$A_1, \ldots, A_m \rightarrow B_1,\ldots,B_n$ and
$B_1,\ldots,B_n \rightarrow C_1,\ldots,C_k$

3. Transitive Closure



$A_1, \ldots, A_m \rightarrow B_1,\ldots,B_n$ and
$B_1,\ldots,B_n \rightarrow C_1,\ldots,C_k$

implies

$A_1,\ldots,A_m \rightarrow C_1,\ldots,C_k$

Finding Functional Dependencies

Example:

Products

Name	Color	Category	Dep	Price
Gizmo	Green	Gadget	Toys	49
Widget	Black	Gadget	Toys	59
Gizmo	Green	Whatsit	Garden	99

Provided FDs:

1. {Name} → {Color}
2. {Category} → {Department}
3. {Color, Category} → {Price}

Which / how many other FDs hold?

Finding Functional Dependencies

Example:

Inferred FDs:

Inferred FD	Rule used
4. {Name, Category} -> {Name}	?
5. {Name, Category} -> {Color}	?
6. {Name, Category} -> {Category}	?
7. {Name, Category -> {Color, Category}	?
8. {Name, Category} -> {Price}	?

Provided FDs:

1. {Name} → {Color}
2. {Category} → {Dept.}
3. {Color, Category} → {Price}

Which / how many other FDs hold?

Finding Functional Dependencies

Example:

Inferred FDs:

Inferred FD	Rule used
4. {Name, Category} -> {Name}	Trivial
5. {Name, Category} -> {Color}	Transitive (4 -> 1)
6. {Name, Category} -> {Category}	Trivial
7. {Name, Category} -> {Color, Category}	Split/Combine (5 + 6)
8. {Name, Category} -> {Price}	Transitive (7 -> 3)

Provided FDs:

1. {Name} → {Color}
2. {Category} → {Dept.}
3. {Color, Category} → {Price}

What's an algorithmic way to do this?

Closures

Closure of a set of Attributes

Given a set of attributes $A_1, \ldots, A_n$ and a set of FDs **F:**
Then the **closure**, $\{A_1, \ldots, A_n\}^+$ is the set of attributes **B** s.t. $\{A_1, \ldots, A_n\} \rightarrow B$

Example: F =

{name} → {color}
{category} → {department}
{color, category} → {price}

Example Closures:

$\{name\}^+ = \{name, color\}$
$\{name, category\}^+ = \{name, category, color, dept, price\}$
$\{color\}^+ = \{color\}$

Closure Algorithm

Start with $X = \{A_1, \ldots, A_n\}$ and set of FDs F.

Repeat until X doesn't change; **do**:

if $\{B_1, \ldots, B_n\} \rightarrow C$ is entailed by F **and** $\{B_1, \ldots, B_n\} \subseteq X$

then add C to X.

Return X as X^+

Closure Algorithm

Start with X = {A_1, …, A_n}, FDs F.
Repeat until X doesn't change; **do**:
  if {B_1, …, B_n} ⟶ C is in F **and** {B_1, …, B_n} ⊆ X:
    then add C to X.
Return X as X^+

F =

{name} ⟶ {color}

{category} ⟶ {dept}

{color, category} ⟶ {price}

{name, category}$^+$ =
{name, category}

Closure Algorithm

Start with X = {A_1, …, A_n}, FDs F.
Repeat until X doesn't change; **do**:
  if {B_1, …, B_n} ⟶ C is in F **and** {B_1, …, B_n} ⊆ X:
    then add C to X.
Return X as X^+

F =

{name} ⟶ {color}

{category} ⟶ {dept}

{color, category} ⟶ {price}

{name, category}$^+$ =
{name, category}

{name, category}$^+$ =
{name, category, color}

Closure Algorithm

Start with X = {A_1, …, A_n}, FDs F.
Repeat until X doesn't change; **do**:
  if {B_1, …, B_n} ⟶ C is in F **and** {B_1, …, B_n} ⊆ X:
    then add C to X.
Return X as X^+

F =

{name} ⟶ {color}

{category} ⟶ {dept}

{color, category} ⟶ {price}

{name, category}$^+$ =
{name, category}

{name, category}$^+$ =
{name, category, color}

{name, category}$^+$ =
{name, category, color, dept}

Closure Algorithm

Start with X = {A_1, …, A_n}, FDs F.
Repeat until X doesn't change; **do**:
  if {B_1, …, B_n} ⟶ C is in F **and** {B_1, …, B_n} ⊆ X:
    then add C to X.
Return X as X^+

F =

{name} ⟶ {color}

{category} ⟶ {dept}

{color, category} ⟶ {price}

{name, category}$^+$ = {name, category}

{name, category}$^+$ = {name, category, color}

{name, category}$^+$ = {name, category, color, dept}

{name, category}$^+$ = {name, category, color, dept, price}

Normalization Cycle

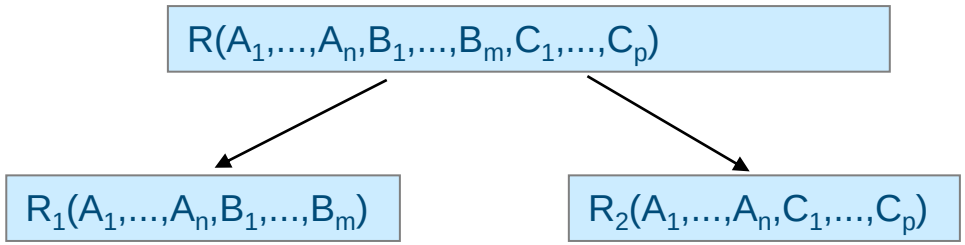
For a “mega” table

- Search for “bad” dependencies
- If any, *keep decomposing the table into sub-tables* until no more bad dependencies
- When done, the database schema is *normalized*

Recall: there are several normal forms…

DECOMPOSITIONS

Decompositions



R_1 = the *projection* of R on A_1, ..., A_n, B_1, ..., B_m

R_2 = the *projection* of R on A_1, ..., A_n, C_1, ..., C_p

Theory of Decomposition

Name	Price	Category
Gizmo	19.99	Gadget
OneClick	24.99	Camera
Gizmo	19.99	Camera

Name	Price
Gizmo	19.99
OneClick	24.99

Name	Category
Gizmo	Gadget
OneClick	Camera
Gizmo	Camera

Sometimes a decomposition is “correct”

i.e. it is a **Lossless decomposition**

Lossy Decomposition

Name	Price	Category
Gizmo	19.99	Gadget
OneClick	24.99	Camera
Gizmo	19.99	Camera

Name	Category
Gizmo	Gadget
OneClick	Camera
Gizmo	Camera

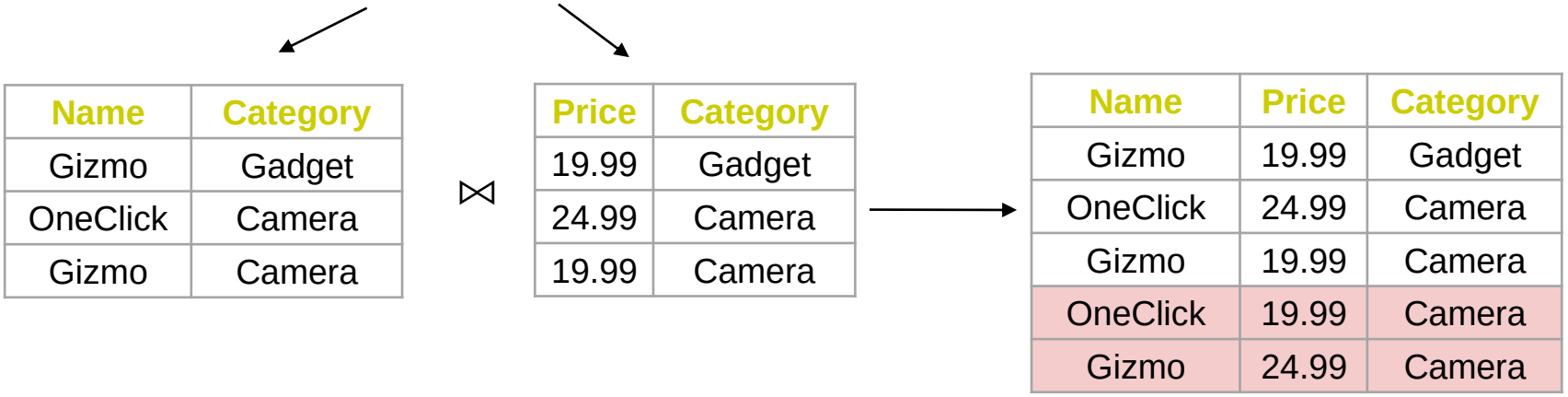
Price	Category
19.99	Gadget
24.99	Camera
19.99	Camera

However sometimes it isn't

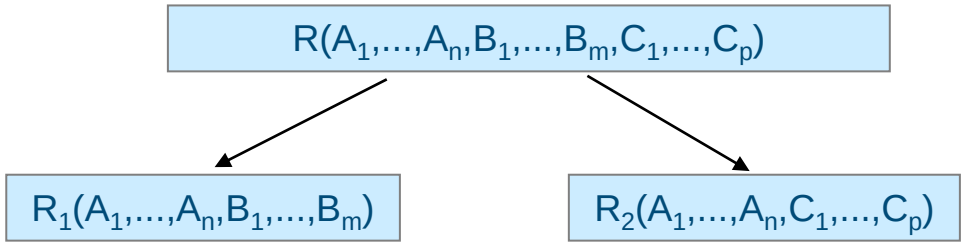
What's wrong here?

Lossy Decomposition

Name	Price	Category
Gizmo	19.99	Gadget
OneClick	24.99	Camera
Gizmo	19.99	Camera

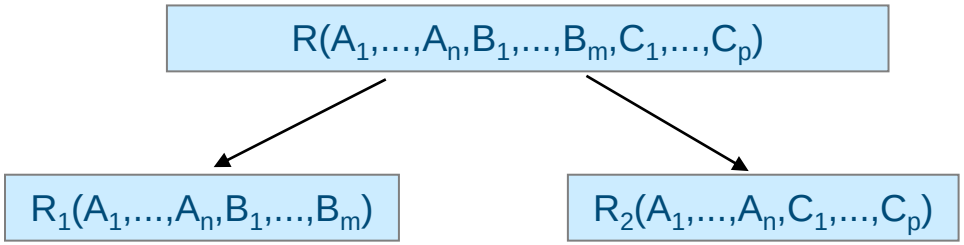


Lossless Decompositions



A decomposition R to (R1, R2) is **lossless** if R = R1 ⋈ R2

Lossless Decompositions



If $\{A_1, ..., A_n\} \rightarrow \{B_1, ..., B_m\}$

or

$\{A_1, ..., A_n\} \rightarrow \{C_1, ..., C_p\}$

Then the decomposition is lossless

A decomposition of R into R_1 and R_2 is lossless decomposition if at least one of the following dependencies is in F^+:

$$R_1 \cap R_2 \rightarrow R_1$$
$$R_1 \cap R_2 \rightarrow R_2$$

Example

- $R = (A, B, C)$
  $F = \{A \rightarrow B, B \rightarrow C\}$
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless decomposition:

    $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
- $R_1 = (A, B)$, $R_2 = (A, C)$
  - Lossless decomposition:

    $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
- $R_1 = (A, C)$, $R_2 = (B, C)$
  - Lossy decomposition:

    $R_1 \cap R_2 = \{C\}$ neither $C \rightarrow AC$ *nor* $C \rightarrow BC$ *holds*
- *Note:*
  - $B \rightarrow BC$ is a shorthand notation for $B \rightarrow \{B, C\}$

Conceptual Design

For a “mega” table

- Search for “bad” dependencies
- If any, *keep decomposing (lossless) the table into sub-tables* until no more bad dependencies
- When done, the database schema is *normalized*

Recall: there are several normal forms…

Normal Forms

- 1st Normal Form (1NF) = All tables are flat
- *2nd Normal Form = disused*
- **Boyce-Codd Normal Form (BCNF)**
- **3rd Normal Form (3NF)**

DB designs based on *functional dependencies*, intended to prevent data ***anomalies***

- *4th and 5th Normal Forms = see text books*

1st Normal Form (1NF)

Student	Courses
Mary	{CS145,CS229}
Joe	{CS145,CS106}
…	…

Violates 1NF.

Student	Courses
Mary	CS145
Mary	CS229
Joe	CS145
Joe	CS106

In 1st NF

1NF Constraint: Types must be atomic!

Boyce-Codd Normal Form

Boyce-Codd Normal Form (BCNF)

Main idea: define "good" and "bad" FDs as follows:

- X → A is a "*good FD" if X is a (super/candidate) key*
  i.e., A is the set of all attributes
- Else, X → A is a *"bad FD"*

- We will try to eliminate the "bad" FDs!

Boyce-Codd Normal Form (BCNF)

Why does this definition of “good” and “bad” FDs make sense?

If X is *not* a (super/candidate)key, it functionally determines *a subset* of the attributes; therefore, those attributes may be duplicated!

Recall: this means there is redundancy and can lead to anomalies

EmpID	Name	Phone	Position
E0045	Smith	1234	Clerk
E3542	Mike	9876	Salesrep
E1111	Smith	9876	Salesrep
E9999	Mary	1234	Lawyer

Example

Name	SSN	PhoneNumber	City
Fred	123-45-6789	206-555-1234	Seattle
Fred	123-45-6789	206-555-6543	Seattle
Joe	987-65-4321	908-555-2121	Westfield
Joe	987-65-4321	908-555-1234	Westfield

{SSN} → {Name,City}

This FD is *bad* because it is **not** a superkey

⇒ **Not** in BCNF

What is the key?
{SSN, PhoneNumber}

Example decomposition

Name	SSN	PhoneNumber	City
Fred	123-45-6789	206-555-1234	Seattle
Fred	123-45-6789	206-555-6543	Seattle
Joe	987-65-4321	908-555-2121	Westfield
Joe	987-65-4321	908-555-1234	Westfield

SSN	PhoneNumber
123-45-6789	206-555-1234
123-45-6789	206-555-6543
987-65-4321	908-555-2121
987-65-4321	908-555-1234

Name	SSN	City
Fred	123-45-6789	Seattle
Joe	987-65-4321	Madison

{SSN} → {Name,City}

This FD is now *good* because it is the key

Now in BCNF!

A Problem with BCNF

Unit	Company	Product
…	…	…

{Unit} → {Company}
{Company,Product} → {Unit}

Unit	Company
…	…

{Unit} → {Company}

Unit	Product
…	…

We do a BCNF decomposition on a “bad” FD:
{Unit} → {Company}

We lose the FD {Company,Product} → {Unit}!!

So Why is that a Problem?

Unit	Company
Galaga99	UW
Bingo	UW

Unit	Product
Galaga99	Databases
Bingo	Databases

{Unit} → {Company}

Unit	Company	Product
Galaga99	UW	Databases
Bingo	UW	Databases

No problem so far. All *local* FD's are satisfied.

Let's put all the data back into a single table again:

Violates the FD {Company,Product} → {Unit}!!

The Problem

- We started with a table R and FDs F
- We decomposed R into BCNF tables R_1, R_2, … with their own FDs F_1, F_2, …
- We insert some tuples into each of the relations—which satisfy their local FDs but when reconstruct it violates some FD **across** tables!

Practical Problem: To enforce FD, must reconstruct R—*on each insert!*

Third Normal Form

- There are some situations where
  - BCNF is not dependency preserving, and
  - efficient checking for FD violation on updates is important
- Solution: define a weaker normal form, called Third Normal Form (3NF)
  - Allows some redundancy (with resultant problems; we will see examples later)
  - But functional dependencies can be checked on individual relations without computing a join.
  - There is always a lossless-join, dependency-preserving decomposition into 3NF.

BCNF vs 3NF

BCNF (recap)

- X → A is a "*good FD" if X is a (super) key*

  I.e., A is the set of all attributes

3NF:

- X → A is a "*good FD" if X is a (super) key*
- Or, if each column in (A – X) is part of some key

BCNF still most common- with additional steps to keep track of lost FDs…

3NF Example

- Consider a schema:

  dept_advisor(s_ID, i_ID, dept_name)

- With functional dependencies:

  $i\_ID \rightarrow dept\_name$

  $s\_ID, dept\_name \rightarrow i\_ID$

- Two candidate keys = {*s_ID, dept_name*}, {*s_ID, i_ID* }
- We note that *dept_advisor* is not in BCNF
- *R*, however, is in 3NF
  - *s_ID, dept_name* is a superkey
  - *i_ID* is NOT a superkey, but:
    - { *dept_name*} – {*i_ID* } = {*dept_name* } and
    - *dept_name* is contained in a candidate key

Comparison of BCNF and 3NF

- It is always possible to decompose a relation into a set of relations that are in 3NF such that:
  - The decomposition is lossless
  - The dependencies are preserved
- It is always possible to decompose a relation into a set of relations that are in BCNF such that:
  - The decomposition is lossless
  - It may not be possible to preserve dependencies.

Comparison of BCNF and 3NF

- Advantages of 3NF over BCNF
  - It is always possible to obtain a 3NF design without sacrificing losslessness or dependency preservation.
- Disadvantages of 3NF
  - We may have to use null values to represent some of the possible meaningful relationships among data items.
  - There is the problem of repetition of information (redundancy).

Goals of Normalization

- Let *R* be a relation schema with a set *F* of functional dependencies.
- Decide whether a relation schema *R* is in "good" form.
- In the case that a relation schema *R* is not in "good" form, decompose it into a set of relation schema $\{R_1, R_2, ..., R_n\}$ such that
  - Each relation schema is in good form
  - The decomposition is a lossless decomposition
  - Preferably, the decomposition should be dependency preserving.

Design Goals

- Goal for a relational database design is:
  - BCNF.
  - Lossless join.
  - Dependency preservation.
- If we cannot achieve this, we accept one of
  - Lack of dependency preservation
  - Redundancy due to use of 3NF
- Interestingly, SQL does not provide a direct way of specifying functional dependencies other than superkeys.

  Can specify FDs using assertions, but they are expensive to test, (and currently not supported by any of the widely used databases!)
- Even if we had a dependency preserving decomposition, using SQL, we would not be able to efficiently test a functional dependency whose left hand side is not a key.